

Retail Inventory and Sales Management System

Design and Implementation of a Database Management System
(RetailDesk: a full-stack application on a normalised MySQL database)

GROUP MEMBERS

#	Name	Roll Number
1	Mukta Shubham	25WU0101080
2	KV susankar Reddy	25WU0101169
3	Nuna Jai Sai Sravan	25WU0101088

COURSE Database Management Systems (DBMS), Project Based Learning

FACULTY Dr. Kiran Mayee Adavala

ACADEMIC YEAR 2026

INSTITUTION Woxsen University

Contents

2. Abstract	. 5
3. Introduction and Problem Statement.	. 6
3.1 Background	6
3.2 Problem statement	. 6
4. Objectives and Scope.	. 7
4.1 Objectives.....	4.27
Scope.....	4.37
Usersofthesystem.	. 7
5. Software and Hardware Requirements.	. 8
5.1 Software requirements.....	8
5.2 Libraries and frameworks.....	8
5.3 Hardware requirements.	. 8
6. ER Diagram.	.10
6.1 Entities and attributes....	10
6.2 Relationships and cardinality	11
6.3 Referential actions chosen.	.11
7. Relational Schema and Normalization.	.12
7.1 Relational schema....	12
7.2 Functional dependencies	12
7.3 Normalization.....	.13
8. Data Dictionary	.14
8.1 Table users (3 rows).....	14
8.2 Table categories(13rows).....	14
8.3 Table suppliers(8rows).....	14
8.4 Table products(32rows).....	15
8.5 Table inventory(32rows).....	15
8.6 Table customers(26rows).....	15
8.7 Table sales (51rows).....	16
8.8 Table sale_items(140rows).....	16
8.9 Table payments(54rows).....	17
8.10 Table stock_movements(179rows)	.17

8.11 Table settings (1 rows)	17
8.12 View v_product_stock.	18
9. SQL Commands Used (DDL, DML) with Sample Outputs.	19
9.1 DDL: database, tables, view and indexes.....	19
9.2 Reference data set (DML: INSERT).....	21
9.3 DML: INSERT, SELECT, UPDATE and DELETE without outputs.....	21
9.4 Integrity constraints in action.....	22
9.5 Transaction: recording a sale (START TRANSACTION/COMMIT/ROLLBACK)	23
10. Queries with Outputs	25
11. UI Design and Screenshots	34
11.1 Design approach.....	34
11.2 Screenshots.....	34
11.2.1 Login.....	34
11.2.2 Dashboard.....	35
11.2.3 Products.....	36
11.2.4 Inventory.....	36
11.2.5 Customers.....	37
11.2.6 New.....	37
11.2.7 Sale (Point of Sale).....	37
11.2.8 Sales.....	38
11.2.9 Tax Invoice.....	39
11.2.10 Payments.....	40
11.2.11 Reports.....	41
11.2.12 Database page: overview.....	42
11.2.13 Database page: ER diagram.....	43
11.2.14 Database page: table schemas.....	44
11.2.15 Database page: SQL queries.....	45
11.2.16 Database page: transactions.....	46
Settings.....	47
12. Implementation Details	48
12.1 Technology stack.....	48
12.2 System architecture.....	48
12.3 Project structure and size.....	49
12.4 Database connectivity.....	49
12.5 Authentication and authorisation.....	50
12.6 The transactional sale (keycode).....	51
12.7 Front-end data layer.....	51

12.8 REST API summary.....	12.9 Security	.52
and data-integritymeasures.		.53
13. Testing (Test Cases and Results)		.54
13.1 Test strategy.....		.54
13.2 Summary of results..		.54
13.3 Test cases andresults		.54
13.4 Additional verification		.58
14. Conclusion and Future Enhancements.		.59
14.1 Conclusion.....	14.2	.59
Limitations.....	14.3 Future	.59
enhancements		.59
15. References		.60
16. Contribution of Each Member		.61
17. Appendix: GitHub Repository Link		.62

Section numbers follow the project-report format prescribed for this course (item 1 is the cover page). Text highlighted [like this] on the cover page, in Section 16 and in Section 17 must be filled in by the group before submission.

2. Abstract

Small and medium retail stores still run on paper registers or loosely connected spreadsheets. Stock counts drift away from reality, invoices are copied by hand, and the owner cannot answer simple questions such as *which products are about to run out* or *who our best customers are*. This project, **RetailDesk**, designs and implements a complete **Retail Inventory and Sales Management System** on top of a carefully normalised relational database. The database, `retail_inventory_db`, runs on **MySQL 8.0 (InnoDB)** and contains **11 tables and 1 view** in Third Normal Form. It enforces integrity with 11 foreign keys (with RESTRICT, CASCADE and SET NULL actions chosen per relationship), UNIQUE and CHECK constraints, a generated column, and indexes on all search paths. A web application (React 18 front end, Node.js and Express back end) lets staff manage products, categories, suppliers, customers and stock; sell through a point-of-sale screen; print GST invoices; track payments; and view dashboards and nine business reports. Every figure on screen is produced by a parameterised SQL query. A sale is written to five tables inside a **single ACID transaction** with row-level locking, so stock can never be oversold or left half-updated.

The system was verified with an automated suite of **111 end-to-end API checks (111 passed, 0 failed)** and 22 documented SQL demonstration queries covering joins, aggregates, subqueries, ROLLUP, a view and a window function. This report documents the requirements, ER design, relational schema and normalisation, data dictionary, SQL commands and query outputs, user interface, implementation, testing, and the planned enhancements.

Keywords: DBMS, MySQL, relational model, normalisation, ER diagram, SQL, ACID transactions, inventory management, point of sale, Node.js, React.

3. Introduction and Problem Statement

3.1 Background

Retail stores such as supermarkets, electronics shops and general stores sell hundreds of products to many customers every day. Each sale changes stock, creates an invoice, records a payment and contributes to the store's revenue. When these records are kept in separate places (a stock register, a bill book and a cash book) the same fact is stored several times and the copies soon disagree. A Database Management System solves this by keeping every fact in exactly one place and by enforcing the rules of the business inside the database itself.

3.2 Problem statement

Design and implement a database management system for a retail store that keeps inventory, suppliers, customers, sales and payments consistent, prevents invalid data and overselling, and turns the stored data into the reports a store manager needs.

The problems observed in a manual or spreadsheet-based store, and how this project addresses each one, are summarised below.

Table 1: Problems of the manual process and the corresponding solution in RetailDesk

Problem in the manual process	Solution in RetailDesk
Stock counts are updated by hand and become wrong; items are sold that are not in stock.	Stock is reduced automatically inside the sale transaction; CHECK (quantity >= 0) and a locking read refuse overselling.
The same supplier, category or customer name is typed repeatedly, with spelling variations.	Normalised tables (3NF) store each fact once and reference it by foreign key.
Deleting a record can leave dangling references (a sale for a deleted product).	Referential actions: RESTRICT, CASCADE and SET NULL are chosen per relationship.
Low-stock items are noticed only when a customer asks for them.	A view with a derived stock status, dashboard alerts and a notification bell highlight items at or below the reorder level.
Totals and invoices are calculated manually and contain errors.	Totals, GST and invoice numbers are calculated on the server; the line subtotal is a generated column.
No quick view of sales trends, best sellers or best customers.	Live dashboard and nine reports built with GROUP BY, JOIN, SUM, COUNT, AVG and HAVING queries.
Anyone can change any record.	JWT login with three roles (Admin, Manager, Cashier) and bcrypt-hashed passwords.

4. Objectives and Scope

4.1 Objectives

- Design a relational database for a retail store and normalise it to **Third Normal Form (3NF)**.
- Draw the ER diagram and map it to a relational schema with primary keys, foreign keys and constraints.
- Implement the schema in **MySQL 8** with tables, a view, indexes, CHECK constraints and a generated column.
- Demonstrate DML and DQL: INSERT, UPDATE, DELETE, joins, aggregates, GROUP BY / HAVING, subqueries, ROLLUP and window functions.
- Demonstrate **transactions (ACID)**: a sale that updates five tables atomically, with locking and a live ROLLBACK demonstration.
- Build a usable web interface: CRUD for master data, a point-of-sale screen, printable invoices, payments, reports and dashboards.
- Validate the system with automated tests and document the results.

4.2 Scope

Table 2: Scope of the project

In scope	Out of scope (see Section 14 for future work)
Single-store retail operation in INR with GST on every sale. Products, categories, suppliers, customers, inventory and stock movements. Point-of-sale billing, invoices, payments (Cash, UPI, Card, Other) and pending-payment follow-up. Dashboard, nine reports with CSV export and print. Role-based access for Admin, Manager and Cashier. An in-app Database page showing the ER diagram, live schema, SQL queries and the ROLLBACK demonstration.	Purchase orders and goods-received notes. Barcode scanning and label printing. Returns and refunds. Batch / expiry tracking. Multi-store inventory. Per-product GST slabs and GST return filing. Integration with a real payment gateway.

4.3 Users of the system

Table 3: System roles

Role	Typical user	Can do
Admin	Store owner	Everything, including store Settings (name, GSTIN, tax rate, thresholds).
Manager	Store manager	Manage catalogue, suppliers, stock adjustments, payments and reports.
Cashier	Billing staff	Create sales at the POS, manage customers and view data; cannot delete catalogue records.

5. Software and Hardware Requirements

5.1 Software requirements

Table4:Software required to run and develop the system

Software	Version	Purpose
Windows / Linux / macOS	Windows 10/11 used for development	Operating system
MySQL Server	8.0 or newer (8.0.46 used)	Relational database (InnoDB). Window functions, CHECK constraints and generated columns need 8.0.
Node.js	18 or newer (LTS)	JavaScript runtime for the API server and the build tools
npm	Bundled with Node.js	Package manager; npm run install:all installs every dependency
Web browser	Any modern browser (Edge, Chrome, Firefox)	Runs the React front end
Visual Studio Code	Any	Code editor (recommended)
MySQL Workbench	Optional	Run the .sql scripts manually and inspect the data

5.2 Libraries and frameworks

Table5:Dependencies declared in package.json files

Layer	Package	Version	Role
Backend	express	^4.22	HTTP server and routing
Backend	mysql2	^3.24	MySQL driver, connection pool, prepared statements
Backend	jsonwebtoken	^9.0	Signing and verifying login tokens (JWT)
Backend	bcryptjs	^3.0	Password hashing
Backend	cors, dotenv	^2.8, ^18.0	Cross-origin access and environment variables
Frontend	react, react-dom	^18.3	User-interface library
Frontend	react-router-dom	^6.30	Client-side routing
Frontend	recharts	^2.15	Line, bar and donut charts
Frontend	lucide-react	^1.52	Icons
Build	vite, @vitejs/plugin-react	^5.4, ^4.7	Development server and production build
Build	tailwindcss, postcss, autoprefixer	^3.4, ^8.5, ^10.6	Utility-first styling
Tooling	concurrently	^10.0	Runs API and web together with one command

5.3 Hardware requirements

The application is lightweight. The figures below are the recommended configuration for a developer machine or a single-counter store; they are guidance rather than measured limits.

Table 6: Hardware requirements

Component	Minimum	Recommended
Processor	Dual-core, 2 GHz	Quad-core, 2.5 GHz or better
Memory (RAM)	4 GB	8 GB or more
Disk space	1 GB free (Node modules, MySQL, project)	5 GB free (room for data growth and backups)
Display	1280 × 720	1920 × 1080 (the interface is responsive down to phone width)
Network	Localhost only	LAN for several billing counters connecting to one server

6. ER Diagram

6.1 Entities and attributes

The ERmodel has eleven entities. The six master entities are **Category**, **Supplier**, **Product**, **Customer**, **User** (staff) and **Settings**. The transaction entities are **Sale** (invoice header), **Sale_Item** (invoice line), **Payment**, **Inventory** (current stock) and **Stock_Movement** (audit trail). Each entity uses a surrogate primary key id (AUTO_INCREMENT) and the natural keys that must stay unique (SKU, invoice number, e-mail, phone) are protected by UNIQUE constraints.

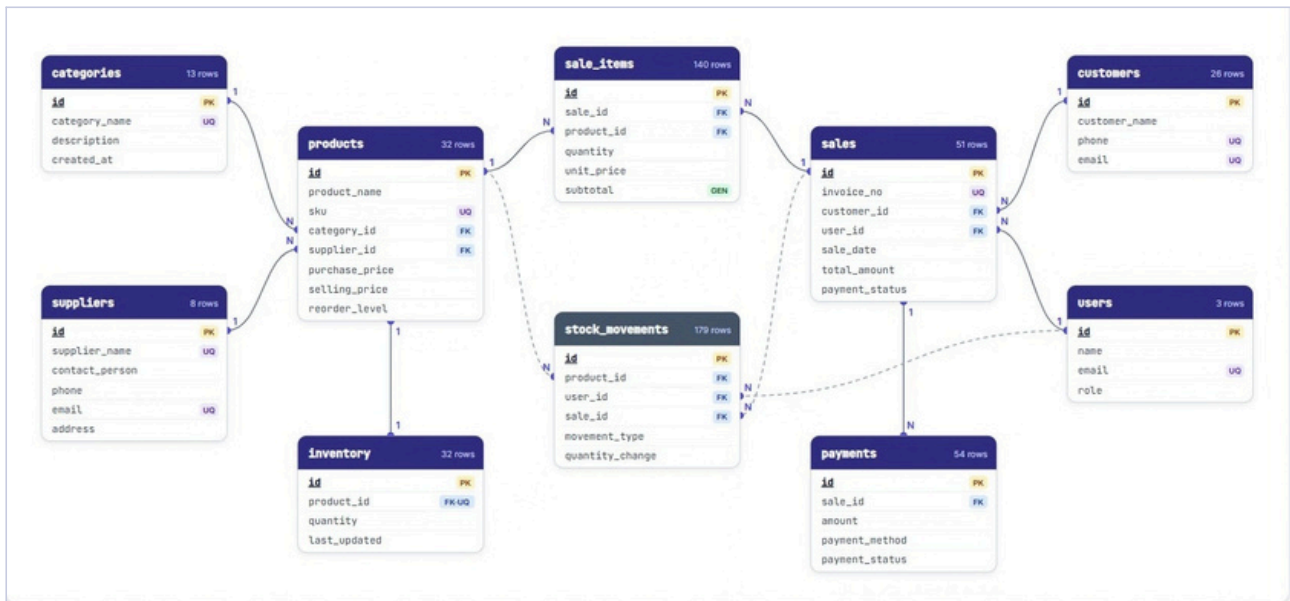


Figure 1: Entity-Relationship diagram of retail_inventory_db (generated live by the application from INFORMATION_SCHEMA; PK = primary key, FK = foreign key, UQ = unique, GEN = generated column; dashed lines are audit relationships)

6.2 Relationships and cardinality

Table 7: Relationships between entities

Parent (1)	Child (N)	Foreign key	Cardinality	ON DELETE	Meaning
products	inventory	product_id	1 : 1	CASCADE	Each product has exactly one stock row (FK is also UNIQUE).
sales	payments	sale_id	1 : N	CASCADE	A sale can have several payment attempts (failed card, then UPI).
categories	products	category_id	1 : N	RESTRICT	A category groups many products; each product has exactly one category.
suppliers	products	supplier_id	1 : N	SET NULL	A supplier supplies many products; the supplier is optional for a product.
products	sale_items	product_id	1 : N	RESTRICT	A product appears on many invoice lines. Together with sale_items this resolves the M:N between Sale and Product.
sales	sale_items	sale_id	1 : N	CASCADE	An invoice has many lines.
customers	sales	customer_id	1 : N	SET NULL	A customer makes many purchases; a sale may be a walk-in (customer NULL).
users	sales	user_id	1 : N	RESTRICT	A staff member bills many sales.
products	stock_movements	product_id	1 : N	CASCADE	Every stock change of a product is logged.
sales	stock_movements	sale_id	1 : N	SET NULL	Which sale caused the change (if any).
users	stock_movements	user_id	1 : N	SET NULL	Who made the change.

6.3 Referential actions chosen

Table 8: Why each ON DELETE rule was chosen

Rule	Applied to	Reason
RESTRICT	products.category_id, sales.user_id, sale_items.product_id	A category still in use, a staff member who billed sales, or a product with sales history must not disappear.
SET NULL	products.supplier_id, sales.customer_id, stock_movements.user_id / sale_id	The child row stays valid without its parent: a product survives if its supplier is removed; sales history is kept as a walk-in sale.
CASCADE	inventory.product_id, sale_items.sale_id, payments.sale_id, stock_movements.product_id	The child has no meaning without the parent (stock row of a deleted product, lines and payments of a deleted sale).

7. Relational Schema and Normalization

7.1 Relational schema

The ER diagram maps to the following relations. Underlined attributes form the primary key, *italic* attributes marked with * are foreign keys, and [U] marks a UNIQUE attribute.

Table 9: Relational schema (11 relations and 1 view)

Relation	Attributes
users	(<u>id</u> , name, email [U], password, role, created_at)
categories	(<u>id</u> , category_name [U], description, created_at)
suppliers	(<u>id</u> , supplier_name [U], contact_person, phone, email [U], address, created_at)
products	(<u>id</u> , product_name, sku [U], <i>category_id*</i> , <i>supplier_id*</i> , description, purchase_price, selling_price, reorder_level, created_at, updated_at)
inventory	(<u>id</u> , <i>product_id*</i> [U], quantity, last_updated)
customers	(<u>id</u> , customer_name, phone [U], email [U], address, created_at)
sales	(<u>id</u> , invoice_no [U], <i>customer_id*</i> , <i>user_id*</i> , sale_date, subtotal, discount, tax, total_amount, payment_status, payment_method, notes)
sale_items	(<u>id</u> , <i>sale_id*</i> , <i>product_id*</i> , quantity, unit_price, subtotal)
payments	(<u>id</u> , <i>sale_id*</i> , payment_date, amount, payment_method, payment_status, transaction_ref)
stock_movements	(<u>id</u> , <i>product_id*</i> , <i>user_id*</i> , <i>sale_id*</i> , movement_type, quantity_change, quantity_after, reason, created_at)
settings	(<u>id</u> , store_name, store_email, store_phone, store_address, gstin, currency, tax_rate, low_stock_threshold, invoice_prefix, updated_at)
v_product_stock (view)	(id, product_name, sku, description, category_id, category_name, supplier_id, supplier_name, purchase_price, selling_price, reorder_level, stock, inventory_value, retail_value, stock_status, last_updated, created_at)

Composite UNIQUE constraint on sale_items (sale_id, product_id) prevents the same product from appearing twice on one invoice; the application merges duplicate cart lines before inserting.

7.2 Functional dependencies

Table 10: Principal functional dependencies

Relation	Functional dependencies (determinant → dependents)
users	id → name, email, password, role, created_at; email → id (candidate key)
categories	id → category_name, description, created_at; category_name → id
suppliers	id → supplier_name, contact_person, phone, email, address; supplier_name → id; email → id
products	id → product_name, sku, category_id, supplier_id, description, purchase_price, selling_price, reorder_level; sku → id
inventory	id → product_id, quantity, last_updated; product_id → id (1:1 with products)
customers	id → customer_name, phone, email, address; phone → id; email → id
sales	id → invoice_no, customer_id, user_id, sale_date, subtotal, discount, tax, total_amount, payment_status, payment_method; invoice_no → id
sale_items	id → sale_id, product_id, quantity, unit_price, subtotal; (sale_id, product_id) → id; (quantity, unit_price) → subtotal (generated)
payments	id → sale_id, payment_date, amount, payment_method, payment_status, transaction_ref
stock_movements	id → product_id, user_id, sale_id, movement_type, quantity_change, quantity_after, reason, created_at
settings	id (always 1) → store_name, store_email, store_phone, store_address, gstin, currency, tax_rate, low_stock_threshold, invoice_prefix

7.3 Normalization

Starting from the un-normalised store ledger (one wide row per invoice line containing the customer, product, category, supplier and stock details), the design was normalised as follows.

Table 11: Normalization steps

Normal form	Rule	How the schema satisfies it
1NF	Every attribute is atomic; no repeating groups.	The products on an invoice are not stored as a list inside sales; each one is a separate row in sale_items. Every column holds a single value.
2NF	No partial dependency of a non-key attribute on part of a composite key.	All tables use a single-column surrogate key, so partial dependencies cannot occur. Product details depend on products.id alone, not on the sale in which the product appears.
3NF	No transitive dependency between non-key attributes.	Category name and supplier details are stored once in categories / suppliers and referenced by foreign key. Customer details are not repeated on every sale. Stock quantity lives in inventory, separate from product master data.

Deliberate design decisions

- **Historical price.** sale_items.unit_price repeats the selling price at the time of sale. This is a historical fact, not redundancy: changing a product's price later must not change old invoices.
- **Generated column.** sale_items.subtotal is a STORED generated column (quantity * unit_price), so it can never disagree with its inputs.
- **Invoice snapshot.** sales.subtotal / tax / total_amount are derivable from the line items and the tax rate. They are stored on purpose as the legal invoice snapshot (the GST rate in Settings may change later) and to make reports fast. They are computed once, on the server, inside the sale transaction.
- **Audit trail.** stock_movements.quantity_after records the stock level after each change, so the history can be read without replaying every movement.

8. Data Dictionary

The data dictionary below was generated from the live database (INFORMATION_SCHEMA.COLUMNS) and completed with the CHECK constraints of schema.sql. Row counts are those of the seeded demonstration data (total 539 rows).

8.1 Table users (3 rows)

#	Column	Data type	Null	Constraints / default	Description
1	id	INT UNSIGNED	NO	PRIMARY KEY AUTO_INCREMENT	Surrogate key
2	name	VARCHAR(100)	NO	–	Full name of the staff member
3	email	VARCHAR(150)	NO	UNIQUE	Login e-mail
4	password	VARCHAR(255)	NO	–	bcrypt hash, never plain text
5	role	ENUM('Admin', 'Manager', 'Cashier')	NO	DEFAULT Cashier	Access level
6	created_at	DATETIME	NO	DEFAULT CURRENT_TIMESTAMP	Account creation time

8.2 Table categories (13 rows)

#	Column	Data type	Null	Constraints / default	Description
1	id	INT UNSIGNED	NO	PRIMARY KEY AUTO_INCREMENT	Surrogate key
2	category_name	VARCHAR(100)	NO	UNIQUE	Category name
3	description	VARCHAR(255)	YES	–	Short description
4	created_at	DATETIME	NO	DEFAULT CURRENT_TIMESTAMP	Creation time

8.3 Table suppliers (8 rows)

#	Column	Data type	Null	Constraints / default	Description
1	id	INT UNSIGNED	NO	PRIMARY KEY AUTO_INCREMENT	Surrogate key
2	supplier_name	VARCHAR(150)	NO	UNIQUE	Vendor name
3	contact_person	VARCHAR(100)	YES	–	Contact person
4	phone	VARCHAR(20)	NO	–	Contact number
5	email	VARCHAR(150)	YES	UNIQUE	Vendor e-mail (optional)
6	address	VARCHAR(255)	YES	–	Postal address
7	created_at	DATETIME	NO	DEFAULT CURRENT_TIMESTAMP	Creation time

8.4 Table products (32 rows)

#	Column	Data type	Null	Constraints / default	Description
1	id	INT UNSIGNED	NO	PRIMARY KEY AUTO_INCREMENT	Surrogate key
2	product_name	VARCHAR(150)	NO	_	Product name Stock-
3	sku	VARCHAR(50)	NO	UNIQUE	keeping unit code
4	category_id	INT UNSIGNED	NO	FK → categories(id) ON DELETE RESTRICT	Category of the product
5	supplier_id	INT UNSIGNED	YES	FK → suppliers(id) ON DELETE SET NULL	Supplier (optional)
6	description	TEXT	YES	_	Free-text description
7	purchase_price	DECIMAL(10,2)	NO	CHECK ≥ 0	Cost price (INR)
8	selling_price	DECIMAL(10,2)	NO	CHECK ≥ 0	Retail price (INR)
9	reorder_level	INT	NO	DEFAULT 10 CHECK ≥ 0	Stock level that triggers a low-stock alert
10	created_at	DATETIME	NO	DEFAULT CURRENT_TIMESTAMP	Creation time
11	updated_at	DATETIME	NO	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Refreshed on every update

8.5 Table inventory (32 rows)

#	Column	Data type	Null	Constraints / default	Description
1	id	INT UNSIGNED	NO	PRIMARY KEY AUTO_INCREMENT	Surrogate key
2	product_id	INT UNSIGNED	NO	UNIQUE FK → products(id) ON DELETE CASCADE	Product this stock row belongs to (1:1)
3	quantity	INT	NO	DEFAULT 0 CHECK ≥ 0	Units currently in stock
4	last_updated	DATETIME	NO	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Refreshed on every change

8.6 Table customers (26 rows)

#	Column	Data type	Null	Constraints / default	Description
1	id	INT UNSIGNED	NO	PRIMARY KEY AUTO_INCREMENT	Surrogate key
2	customer_name	VARCHAR(100)	NO	_	Customer name
3	phone	VARCHAR(20)	NO	UNIQUE	Mobile number
4	email	VARCHAR(150)	YES	UNIQUE	E-mail (optional)
5	address	VARCHAR(255)	YES	_	Postal address
6	created_at	DATETIME	NO	DEFAULT CURRENT_TIMESTAMP	Creation time

8.7 Table sales (51 rows)

#	Column	Data type	Null	Constraints / default	Description
1	id	INT UNSIGNED	NO	PRIMARY KEY AUTO_INCREMENT	Surrogate key
2	invoice_no	VARCHAR(30)	YES	UNIQUE	Invoice number, e.g. INV-000051
3	customer_id	INT UNSIGNED	YES	FK → customers(id) ON DELETE SET NULL	Buyer (NULL = walk-in)
4	user_id	INT UNSIGNED	NO	FK → users(id) ON DELETE RESTRICT	Staff member who billed the sale
5	sale_date	DATETIME	NO	DEFAULT CURRENT_TIMESTAMP	Date and time of sale
6	subtotal	DECIMAL(12,2)	NO	DEFAULT 0.00	Sum of line amounts
7	discount	DECIMAL(12,2)	NO	CHECK ≥ 0 DEFAULT 0.00	Discount given (INR)
8	tax	DECIMAL(12,2)	NO	CHECK ≥ 0; CHECK discount ≤ subtotal DEFAULT 0.00	GST amount
9	total_amount	DECIMAL(12,2)	NO	CHECK ≥ 0 DEFAULT 0.00	subtotal – discount + tax
10	payment_status	ENUM('Paid', 'Pending', 'Failed')	NO	CHECK ≥ 0 DEFAULT Paid	Paid / Pending / Failed
11	payment_method	ENUM('Cash', 'UPI', 'Card', 'Other')	NO	DEFAULT Cash	Cash / UPI / Card / Other
12	notes	VARCHAR(255)	YES	–	Optional note

8.8 Table sale_items (140 rows)

#	Column	Data type	Null	Constraints / default	Description
1	id	INT UNSIGNED	NO	PRIMARY KEY AUTO_INCREMENT	Surrogate key
2	sale_id	INT UNSIGNED	NO	UNIQUE (sale_id, product_id) FK → sales(id) ON DELETE CASCADE	Invoice the line belongs to
3	product_id	INT UNSIGNED	NO	UNIQUE (sale_id, product_id) FK → products(id) ON DELETE RESTRICT	Product sold
4	quantity	INT	NO	CHECK > 0	Units sold
5	unit_price	DECIMAL(10,2)	NO	CHECK ≥ 0	Price at the time of sale
6	subtotal	DECIMAL(12,2)	YES	GENERATED: (quantity * unit_price) STORED	quantity × unit_price (generated)

8.9 Table payments (54 rows)

#	Column	Data type	Null	Constraints / default	Description
1	id	INT UNSIGNED	NO	PRIMARY KEY AUTO_INCREMENT	Surrogate key
2	sale_id	INT UNSIGNED	NO	FK → sales(id) ON DELETE CASCADE	Sale being paid
3	payment_date	DATETIME	NO	DEFAULT CURRENT_TIMESTAMP	Date and time of attempt
4	amount	DECIMAL(12,2)	NO	CHECK > 0	Amount paid (INR)
5	payment_method	ENUM('Cash', 'UPI', 'Card', 'Other')	NO	—	Cash / UPI / Card / Other
6	payment_status	ENUM('Paid', 'Pending', 'Failed')	NO	DEFAULT Paid	Paid / Pending / Failed
7	transaction_ref	VARCHAR(50)	YES	—	Gateway / UPI / card reference

8.10 Table stock_movements (179 rows)

#	Column	Data type	Null	Constraints / default	Description
1	id	INT UNSIGNED	NO	PRIMARY KEY AUTO_INCREMENT	Surrogate key
2	product_id	INT UNSIGNED	NO	FK → products(id) ON DELETE CASCADE	Product whose stock changed
3	user_id	INT UNSIGNED	YES	FK → users(id) ON DELETE SET NULL	Staff member (NULL = system)
4	sale_id	INT UNSIGNED	YES	FK → sales(id) ON DELETE SET NULL	Related sale (if any)
5	movement_type	ENUM('INITIAL', 'SALE', 'RESTOCK', 'ADJUSTMENT')	NO	—	INITIAL / SALE / RESTOCK / ADJUSTMENT
6	quantity_change	INT	NO	—	Signed change in units
7	quantity_after	INT	NO	CHECK ≥ 0	Stock after the change
8	reason	INT	YES	—	Reason or invoice reference
9	created_at	DATETIME	NO	DEFAULT CURRENT_TIMESTAMP	Time of change

8.11 Table settings (1 rows)

#	Column	Data type	Null	Constraints / default	Description
1	id	TINYINT UNSIGNED	NO	PRIMARY KEY DEFAULT 1 CHECK id = 1	Always 1 (single-row table)
2	store_name	VARCHAR(150)	NO	—	Store name printed on invoices
3	store_email	VARCHAR(150)	YES	—	Store e-mail
4	store_phone	VARCHAR(20)	YES	—	Store phone
5	store_address	VARCHAR(255)	YES	—	Store address
6	gst_in	VARCHAR(20)	YES	—	GST identification number
7	currency	VARCHAR(10)	NO	DEFAULT INR	Currency code
8	tax_rate	DECIMAL(5,2)	NO	DEFAULT 18.00 CHECK 0..100	GST percentage applied to sales
9	low_stock_threshold	INT	NO	DEFAULT 10 CHECK ≥ 0	Default low-stock level
10	invoice_prefix	VARCHAR(10)	NO	DEFAULT INV	Prefix for invoice numbers

#	Column	Data type	Null	Constraints / default	Description
11	updated_at	DATETIME	NO	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Refreshed on every update

8.12 View v_product_stock

Onerow perproduct,joining products, categories, suppliers and inventory. Besides the base columns it derives stock (COALESCE of quantity, 0), inventory_value (stock × purchase price), retail_value (stock × selling price) and stock_status (*Out of Stock* when 0, *Low Stock* when at or below the reorder level, otherwise *In Stock*). It is used by the Products, Inventory and Dashboard screens.

9. SQL Commands Used (DDL, DML) with Sample Outputs

9.1 DDL: database, tables, view and indexes

The complete DDL is in database/schema.sql (comments removed below to save space). It is executed by `npm run setup-db`, which drops and recreates the database, then loads seed.sql.

```
DROP DATABASE IF EXISTS retail_inventory_db;
CREATE DATABASE retail_inventory_db CHARACTER SET utf8mb4 COLLATE utf8mb4_unicode_ci;
USE retail_inventory_db;

CREATE TABLE users (
  id          INT UNSIGNED AUTO_INCREMENT,
  name        VARCHAR(100) NOT NULL,
  email       VARCHAR(150) NOT NULL,
  password    VARCHAR(255) NOT NULL COMMENT 'bcrypt hash, never plain text',
  role        ENUM('Admin', 'Manager', 'Cashier') NOT NULL DEFAULT 'Cashier',
  created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT pk_users PRIMARY KEY (id),
  CONSTRAINT uq_users_email UNIQUE (email)
) ENGINE = InnoDB;

CREATE TABLE categories (
  id          INT UNSIGNED AUTO_INCREMENT,
  category_name VARCHAR(100) NOT NULL,
  description  VARCHAR(255) NULL,
  created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT pk_categories PRIMARY KEY (id),
  CONSTRAINT uq_categories_name UNIQUE (category_name)
) ENGINE = InnoDB;

CREATE TABLE suppliers (
  id          INT UNSIGNED AUTO_INCREMENT,
  supplier_name VARCHAR(150) NOT NULL,
  contact_person VARCHAR(100) NULL,
  phone_email  VARCHAR(20) NOT NULL,
  address      VARCHAR(150) NULL,
  created_at  VARCHAR(255) NULL,
  created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT pk_suppliers PRIMARY KEY (id),
  CONSTRAINT uq_suppliers_name UNIQUE (supplier_name),
  CONSTRAINT uq_suppliers_email UNIQUE (email)
) ENGINE = InnoDB;

CREATE TABLE products (
  id          INT UNSIGNED AUTO_INCREMENT,
  product_name VARCHAR(150) NOT NULL,
  sku         VARCHAR(50) NOT NULL,
  category_id INT UNSIGNED NOT NULL,
  supplier_id INT UNSIGNED NULL,
  description  TEXT NULL,
  purchase_price DECIMAL(10, 2) NOT NULL,
  selling_price DECIMAL(10, 2) NOT NULL,
  reorder_level INT NOT NULL DEFAULT 10,
  created_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  updated_at  DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT pk_products PRIMARY KEY (id),
  CONSTRAINT uq_products_sku UNIQUE (sku),
  CONSTRAINT fk_products_category FOREIGN KEY (category_id)
    REFERENCES categories (id) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT fk_products_supplier FOREIGN KEY (supplier_id)
    REFERENCES suppliers (id) ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT chk_products_purchase_price CHECK (purchase_price >= 0),
  CONSTRAINT chk_products_selling_price CHECK (selling_price >= 0),
  CONSTRAINT chk_products_reorder_level CHECK (reorder_level >= 0),
  INDEX idx_products_name (product_name)
) ENGINE = InnoDB;

CREATE TABLE inventory (
  id          INT UNSIGNED AUTO_INCREMENT,
  product_id  INT UNSIGNED NOT NULL,
  quantity    INT NOT NULL DEFAULT 0,
  last_updated DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT pk_inventory PRIMARY KEY (id),
  CONSTRAINT uq_inventory_product UNIQUE (product_id),
  CONSTRAINT fk_inventory_product FOREIGN KEY (product_id)
    REFERENCES products (id) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT chk_inventory_quantity CHECK (quantity >= 0)
) ENGINE = InnoDB;

CREATE TABLE customers (
  id          INT UNSIGNED AUTO_INCREMENT,
```

```

customer_name VARCHAR(100) NOT NULL,
phone_email   VARCHAR(20) NOT NULL,
address        VARCHAR(150) NULL,
created_at     VARCHAR(255) NULL,
               DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
CONSTRAINT pk_customers PRIMARY KEY (id),
CONSTRAINT uq_customers_phone UNIQUE (phone),
CONSTRAINT uq_customers_email UNIQUE (email),
INDEX idx_customers_name (customer_name)
) ENGINE = InnoDB;

CREATE TABLE sales (
  id invoice_no INT UNSIGNED AUTO_INCREMENT,
  customer_id   VARCHAR(30) NULL,
  user_id       INT UNSIGNED NULL,
  sale_date     INT UNSIGNED NOT NULL,
  subtotal      DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  discount      DECIMAL(12, 2) NOT NULL DEFAULT 0,
  tax           DECIMAL(12, 2) NOT NULL DEFAULT 0,
  total_amount  DECIMAL(12, 2) NOT NULL DEFAULT 0,
               DECIMAL(12, 2) NOT NULL DEFAULT 0,
  payment_status ENUM('Paid', 'Pending', 'Failed') NOT NULL DEFAULT 'Paid',
  payment_method ENUM('Cash', 'UPI', 'Card', 'Other') NOT NULL DEFAULT 'Cash',
  notes         VARCHAR(255) NULL,
  CONSTRAINT pk_sales PRIMARY KEY (id),
  CONSTRAINT uq_sales_invoice UNIQUE (invoice_no),
  CONSTRAINT fk_sales_customer FOREIGN KEY (customer_id)
    REFERENCES customers (id) ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT fk_sales_user FOREIGN KEY (user_id)
    REFERENCES users (id) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT chk_sales_amounts CHECK (subtotal >= 0 AND discount >= 0 AND tax >= 0 AND total_amount >= 0),
  CONSTRAINT chk_sales_discount CHECK (discount <= subtotal),
  INDEX idx_sales_date (sale_date),
  INDEX idx_sales_status (payment_status)
) ENGINE = InnoDB;

CREATE TABLE sale_items (
  id INT UNSIGNED AUTO_INCREMENT,
  sale_id INT UNSIGNED NOT NULL,
  product_id INT UNSIGNED NOT NULL,
  quantity INT NOT NULL,
  unit_price DECIMAL(10, 2) NOT NULL,
  subtotal DECIMAL(12, 2) AS (quantity * unit_price) STORED,
  CONSTRAINT pk_sale_items PRIMARY KEY (id),
  CONSTRAINT uq_sale_items_sale_product UNIQUE (sale_id, product_id),
  CONSTRAINT fk_sale_items_sale FOREIGN KEY (sale_id)
    REFERENCES sales (id) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_sale_items_product FOREIGN KEY (product_id)
    REFERENCES products (id) ON DELETE RESTRICT ON UPDATE CASCADE,
  CONSTRAINT chk_sale_items_quantity CHECK (quantity > 0),
  CONSTRAINT chk_sale_items_price CHECK (unit_price >= 0)
) ENGINE = InnoDB;

CREATE TABLE payments (
  id INT UNSIGNED AUTO_INCREMENT,
  sale_id INT UNSIGNED NOT NULL,
  payment_date DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  amount DECIMAL(12, 2) NOT NULL,
  payment_method ENUM('Cash', 'UPI', 'Card', 'Other') NOT NULL,
  payment_status ENUM('Paid', 'Pending', 'Failed') NOT NULL DEFAULT 'Paid',
  transaction_ref VARCHAR(50) NULL,
  CONSTRAINT pk_payments PRIMARY KEY (id),
  CONSTRAINT fk_payments_sale FOREIGN KEY (sale_id)
    REFERENCES sales (id) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT chk_payments_amount CHECK (amount > 0),
  INDEX idx_payments_status (payment_status)
) ENGINE = InnoDB;

CREATE TABLE stock_movements (
  id INT UNSIGNED AUTO_INCREMENT,
  product_id INT UNSIGNED NOT NULL,
  user_id INT UNSIGNED NULL,
  sale_id INT UNSIGNED NULL,
  movement_type ENUM('INITIAL', 'SALE', 'RESTOCK', 'ADJUSTMENT') NOT NULL,
  quantity_change INT NOT NULL,
  quantity_after INT NOT NULL,
  reason VARCHAR(255) NULL,
  created_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT pk_stock_movements PRIMARY KEY (id),
  CONSTRAINT fk_movements_product FOREIGN KEY (product_id)
    REFERENCES products (id) ON DELETE CASCADE ON UPDATE CASCADE,
  CONSTRAINT fk_movements_user FOREIGN KEY (user_id)
    REFERENCES users (id) ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT fk_movements_sale FOREIGN KEY (sale_id)
    REFERENCES sales (id) ON DELETE SET NULL ON UPDATE CASCADE,
  CONSTRAINT chk_movements_after CHECK (quantity_after >= 0),
  INDEX idx_movements_created (created_at)
) ENGINE = InnoDB;

```

```

CREATE TABLE settings (
  id TINYINT UNSIGNED NOT NULL DEFAULT 1,
  store_name VARCHAR(150) NOT NULL,
  store_email VARCHAR(150) NULL,
  store_phone VARCHAR(20) NULL,
  store_address VARCHAR(255) NULL,
  gstin_currency VARCHAR(20) NULL,
  tax_rate VARCHAR(10) NOT NULL DEFAULT 'INR',
  DECIMAL(5, 2) NOT NULL DEFAULT 18.00,

  low_stock_threshold INT NOT NULL DEFAULT 10,
  invoice_prefix VARCHAR(10) NOT NULL DEFAULT 'INV',
  updated_at DATETIME NOT NULL DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP,
  CONSTRAINT pk_settings PRIMARY KEY (id),
  CONSTRAINT chk_settings_single_row CHECK (id = 1),
  CONSTRAINT chk_settings_tax CHECK (tax_rate BETWEEN 0 AND 100),
  CONSTRAINT chk_settings_threshold CHECK (low_stock_threshold >= 0)
) ENGINE = InnoDB;

CREATE OR REPLACE VIEW v_product_stock AS
SELECT
  p.id,
  p.product_name,
  p.sku,
  p.description,
  p.category_id,
  c.category_name,
  p.supplier_id,
  s.supplier_name,
  p.purchase_price,
  p.selling_price,
  p.reorder_level,
  COALESCE(i.quantity, 0) AS stock,
  COALESCE(i.quantity, 0) * p.purchase_price AS inventory_value,
  COALESCE(i.quantity, 0) * p.selling_price AS retail_value,
  CASE
    WHEN COALESCE(i.quantity, 0) = 0 THEN 'Out of Stock'
    WHEN i.quantity <= p.reorder_level THEN 'Low Stock'
    ELSE 'In Stock'
  END AS stock_status,
  i.last_updated,
  p.created_at
FROM products p
INNER JOIN categories c ON c.id = p.category_id
LEFT JOIN suppliers s ON s.id = p.supplier_id
LEFT JOIN inventory i ON i.product_id = p.id;

```

9.2 Reference data set (DML: INSERT)

The demonstration data in seed.sql is loaded with multi-row INSERT statements (dates are relative to **NOW()** so the dashboard always shows recent activity). Example:

```

INSERT INTO categories (id, category_name, description, created_at) VALUES
(1, 'Groceries', 'Staples such as rice, pulses, flour, oil and salt', DATE_SUB(NOW(), INTERVAL 110 DAY)),
(2, 'Snacks', 'Biscuits, chips, namkeen and dry fruits', DATE_SUB(NOW(), INTERVAL 109 DAY)), ... ;

```

Table 12: Rows loaded by seed.sql (counts verified in the running database)

Table	users	categories	suppliers	products	inventory	customers
Rows	3	13	8	32	32	26

Table	sales	sale_items	payments	stock_movements	settings
Rows	51	140	54	179	1

9.3 DML: INSERT, SELECT, UPDATE and DELETE with outputs

The statements of database/crud_examples.sql were executed against the live database **inside a transaction that was rolled back afterwards**, so the outputs below are real while the sample data stayed untouched.

INSERT category

```

INSERT INTO categories (category_name, description)
VALUES ('Frozen Foods', 'Frozen snacks, peas and ready-to-cook items');

```

Query OK, 1 row(s) affected (insertId 14)

INSERT product

```
INSERT INTO products (product_name, sku, category_id, supplier_id, purchase_price, selling_price, reorder_level)
VALUES ('Frozen Green Peas 500g', 'FRZ-PEAS-500', 14, 6, 55.00, 80.00, 10);
```

Query OK, 1 row(s) affected (insertId 33)

INSERT inventory

```
INSERT INTO inventory (product_id, quantity)
VALUES (33, 40);
```

Query OK, 1 row(s) affected (insertId 33)

SELECT view

```
SELECT product_name, sku, category_name, supplier_name, selling_price, stock, stock_status
FROM v_product_stock
WHERE sku = 'FRZ-PEAS-500';
```

product_name	sku	category_name	supplier_name	selling_price	stock	stock_status
Frozen Green Peas 500g	FRZ-PEAS-500	Frozen Foods	Metro FMCG Wholesale	80.00	40	In Stock

UPDATE price

```
UPDATE products
SET selling_price = 85.00
WHERE sku = 'FRZ-PEAS-500';
```

Query OK, 1 row(s) affected

UPDATE restock

```
UPDATE inventory i
JOIN products p ON p.id = i.product_id
SET i.quantity = i.quantity + 20
WHERE p.sku = 'FRZ-PEAS-500';
```

Query OK, 1 row(s) affected

SELECT after update

```
SELECT product_name, selling_price, stock, stock_status
FROM v_product_stock
WHERE sku = 'FRZ-PEAS-500';
```

product_name	selling_price	stock	stock_status
Frozen Green Peas 500g	85.00	60	In Stock

DELETE product

```
DELETE
FROM products
WHERE sku = 'FRZ-PEAS-500';
```

Query OK, 1 row(s) affected

inventory cascaded

```
SELECT COUNT(*) AS inventory_rows_left
FROM inventory
WHERE product_id = 33;
```

inventory_rows_left
0

DELETE category

```
DELETE
FROM categories
WHERE category_name = 'Frozen Foods';
```

Query OK, 1 row(s) affected

9.4 Integrity constraints in action

Each statement below violates a rule on purpose; MySQL rejects it and nothing is changed. The messages are the real server responses.

Table 13: Constraint violations and the errors returned by MySQL 8.0.46

Rule tested	Statement	MySQL response
Duplicate SKU (UNIQUE)	INSERT INTO products (product_name, sku, category_id, purchase_price, selling_price) VALUES ('Duplicate', 'GRO-RICE-5KG', 1, 10, 20)	ERROR 1062 (23000): Duplicate entry 'GRO-RICE-5KG' for key 'products.uq_products_sku'
Negative price (CHECK)	UPDATE products SET selling_price = -5 WHERE id = 1	ERROR 3819 (HY000): Check constraint 'chk_products_selling_price' is violated.
Delete category in use (FK RESTRICT)	DELETE FROM categories WHERE category_name = 'Groceries'	ERROR 1451 (23000): Cannot delete or update a parent row: a foreign key constraint fails ('retail_inventory_db'.products, CONSTRAINT 'fk_products_category' FOREIGN KEY ('category_id') REFERENCES categories ('id') ON DELETE RESTRICT ON UPDATE CASCADE)
Negative stock (CHECK)	UPDATE inventory SET quantity = quantity - 100000 WHERE product_id = 1	ERROR 3819 (HY000): Check constraint 'chk_inventory_quantity' is violated.
Discount greater than subtotal (CHECK)	INSERT INTO sales (user_id, subtotal, discount, tax, total_amount) VALUES (1, 100, 500, 0, 0)	ERROR 3819 (HY000): Check constraint 'chk_sales_discount' is violated.

9.5 Transaction: recording a sale (START TRANSACTION / COMMIT / ROLLBACK)

Asaletouches sales, sale_items, inventory and payments (and logs stock_movements). All steps must succeed together or not at all.

```

-----
-- SALE TRANSACTION (what POST /api/sales does)
-- All five steps succeed together or are rolled back together.
-----
START TRANSACTION;

-- 1. lock the stock rows being sold so concurrent sales cannot oversell
SELECT product_id, quantity FROM inventory WHERE product_id IN (1, 6) FOR UPDATE;

-- 2. sale header (customer 1, billed by user 1)
INSERT INTO sales (customer_id, user_id, subtotal, discount, tax, total_amount, payment_status, payment_method)
VALUES (1, 1, 739.00, 0.00, 133.02, 872.02, 'Paid', 'UPI');
SET @sale_id = LAST_INSERT_ID();
UPDATE sales SET invoice_no = CONCAT('INV-', LPAD(@sale_id, 6, '0')) WHERE id = @sale_id;

-- 3. line items (subtotal column is generated automatically)
INSERT INTO sale_items (sale_id, product_id, quantity, unit_price) VALUES
(@sale_id, 1, 1, 549.00),
(@sale_id, 6, 2, 95.00);

-- 4. reduce stock (the WHERE guard refuses to oversell)
UPDATE inventory SET quantity = quantity - 1 WHERE product_id = 1 AND quantity >= 1;
UPDATE inventory SET quantity = quantity - 2 WHERE product_id = 6 AND quantity >= 2;

-- 5. payment record
INSERT INTO payments (sale_id, amount, payment_method, payment_status)
VALUES (@sale_id, 872.02, 'UPI', 'Paid');

COMMIT;    -- or ROLLBACK; to undo every step above

```

Table 14: Result of running the sale transaction and then issuing ROLLBACK (stock of "Premium Basmati Rice 5kg")

Moment	Stock of product #1	Comment
Before START TRANSACTION	48	Original stock
After the UPDATE inside the transaction	47	Visible only inside this transaction (1 unit sold)
After ROLLBACK	48	Original value restored; sales table still has 51 rows

Line items inserted by the transaction (generated column subtotal computed by MySQL):

product_name invoice_no	quantity	unit_price	subtotal
Premium Basmati Rice 5kg INV-000052	1	549.00	549.00
Glucose Biscuits 800g INV-000052	2	95.00	190.00

With **COMMIT** instead of ROLLBACK the invoice, items, payment and the stock reduction would become permanent together. The Database page of the application contains a live ROLLBACK demonstration (Figure 16 in Section 11).

10. Queries with Outputs

The 22 demonstration queries below are stored in `database/queries.sql` and are also runnable from the application (Database → SQL Queries) in a READ ONLY transaction. The outputs shown here were produced by running each query on the seeded database on **5 October 2026**. Where a result is long, the first rows are shown and the row count is given. Queries that use `CURDATE()` depend on the date of execution.

Table 15: Index of queries and the DBMS concepts they demonstrate

#	Query	Concepts
1	Products with category names	SELECT, INNER JOIN, ORDER BY
2	Products with supplier names	LEFT JOIN, COALESCE
3	Products at or below reorder level	INNER JOIN, WHERE, CASE
4	Overall sales summary	COUNT, SUM, AVG, MIN, MAX
5	Today's sales	Date filtering, <code>CURDATE()</code> , LEFT JOIN
6	Top 10 selling products (by quantity)	JOIN, GROUP BY, SUM, ORDER BY, LIMIT
7	Monthly sales	GROUP BY, <code>DATE_FORMAT</code> , COUNT, SUM
8	Total sales by category	Multi-table JOIN, GROUP BY, SUM
9	Customers with the highest purchase amounts	JOIN, GROUP BY, COUNT, SUM, ORDER BY
10	Total inventory value (with ROLLUP)	SUM, GROUP BY ... WITH ROLLUP, <code>GROUPING()</code>
11	Products that have never been sold	Subquery, NOT EXISTS
12	Best-selling products (HAVING filter)	GROUP BY, HAVING, SUM, Profit calculation
13	Sales with customer and cashier details	LEFT JOIN, INNER JOIN, Correlated subquery
14	Total quantity sold per product (including zero)	LEFT JOIN, COALESCE, GROUP BY
15	Suppliers supplying the most products	LEFT JOIN, GROUP BY, COUNT, SUM
16	Customers who spend more than the average customer	Nested subquery, Derived table, HAVING
17	Payment method & status summary	GROUP BY multiple columns, COUNT, SUM
18	Daily sales for the last 7 days	Date filtering, BETWEEN, <code>DATE_SUB</code> , GROUP BY
19	Querying a VIEW (<code>v_product_stock</code>)	VIEW, CASE, Derived columns
20	Categories with product counts (including empty)	LEFT JOIN, GROUP BY, COUNT
21	Rank products by revenue within each category	Window function, <code>RANK()</code> OVER (<code>PARTITION BY</code>)
22	Stock movement audit trail	JOIN, LEFT JOIN, ORDER BY, LIMIT

10.1 Products with category names

Concepts: SELECT, INNER JOIN, ORDER BY

Every product together with the name of the category it belongs to. INNER JOIN returns only products whose `category_id` matches a row in categories.

```
SELECT p.id, p.product_name, p.sku, c.category_name, p.selling_price
FROM products p
INNER JOIN categories c ON p.category_id = c.id
ORDER BY c.category_name, p.product_name;
```

Output: 32 rows (first 6 shown)

id	product_name	sku	category_name	selling_price
11	Assam Tea 500g	BEV-TEA-500G	Beverages	275.00
9	Cola Soft Drink 2L	BEV-COLA-2L	Beverages	95.00
12	Instant Coffee 100g	BEV-COFF-100G	Beverages	340.00
10	Mango Fruit Drink 1.2L	BEV-MANGO-1L	Beverages	75.00
18	Detergent Powder 2kg	CLN-DET-2KG	Cleaning Products	250.00
17	Dishwash Gel 750ml	CLN-DISH-750	Cleaning Products	149.00

10.2 Products with supplier names

Concepts: LEFT JOIN, COALESCE

LEFT JOIN keeps a product even if its supplier was deleted (supplier_id is set to NULL by ON DELETE SET NULL).

```
SELECT p.product_name, p.sku, COALESCE(s.supplier_name, '- No supplier -') AS supplier_name,
       s.contact_person, s.phone
FROM products p
LEFT JOIN suppliers s ON p.supplier_id = s.id
ORDER BY supplier_name, p.product_name;
```

Output: 32 rows (first 6 shown)

product_name	sku	supplier_name	contact_person	phone
Assam Tea 500g	BEV-TEA-500G	Annapurna Foods Pvt Ltd	Sunita Joshi	9823045678
Instant Coffee 100g	BEV-COFF-100G	Annapurna Foods Pvt Ltd	Sunita Joshi	9823045678
Refined Sunflower Oil 1L	GRO-OIL-1L	Annapurna Foods Pvt Ltd	Sunita Joshi	9823045678
Salted Cashews 250g	SNK-CASH-250G	Annapurna Foods Pvt Ltd	Sunita Joshi	9823045678
Whole Wheat Atta 10kg	GRO-ATTA-10KG	Annapurna Foods Pvt Ltd	Sunita Joshi	9823045678
Electric Kettle 1.5L	HA-KETL-1.5L	Bharat Electricals	Jignesh Shah	9824078901

10.3 Products at or below reorder level

Concepts: INNER JOIN, WHERE, CASE

Products whose current stock (inventory table) has fallen to or below their reorder level — the store's restocking list.

```
SELECT p.product_name, p.sku, i.quantity AS current_stock, p.reorder_level,
       CASE WHEN i.quantity = 0 THEN 'Out of Stock' ELSE 'Low Stock' END AS status,
       (p.reorder_level * 2 - i.quantity) AS suggested_order_qty
FROM products p
INNER JOIN inventory i ON i.product_id = p.id
WHERE i.quantity <= p.reorder_level
ORDER BY i.quantity ASC;
```

Output: 8 rows (first 6 shown)

product_name	sku	current_stock	reorder_level	status	suggested_order_qty
Masala Potato Chips 90g	SNK-CHIP-90G	0	30	Out of Stock	60
Dishwash Gel 750ml	CLN-DISH-750	0	15	Out of Stock	30
Non-stick Frying Pan 24cm	KIT-FPAN-24	3	5	Low Stock	7
Optical USB Mouse	CMP-MOUS-USB	6	10	Low Stock	14
Gel Pen Set (10 pcs)	STN-PEN-10PK	7	20	Low Stock	33
Fluoride Toothpaste 150g	PC-TPST-150	9	25	Low Stock	41

10.4 Overall sales summary

Concepts: COUNT, SUM, AVG, MIN, MAX

Aggregate functions over the whole sales table — number of orders, total revenue, and the average, smallest and largest bill.

```
SELECT COUNT(*)           AS total_orders,
       SUM(total_amount)  AS total_revenue,
       ROUND(AVG(total_amount), 2) AS average_order_value,
       MIN(total_amount)  AS smallest_order,
       MAX(total_amount)  AS largest_order,
       SUM(discount)      AS total_discount_given,
       SUM(tax)           AS total_tax_collected
FROM sales;
```

Output: 1 row

total_orders	total_revenue	average_order_value	smallest_order	largest_order	total_discount_given	total_tax_collected
51	80332.04	1575.14	233.64	5005.56	783.00	12254.04

10.5 Today's sales

Concepts: Datefiltering, CURDATE(), LEFT JOIN

All invoices created today, using DATE(sale_date) = CURDATE().

```
SELECT s.invoice_no, TIME(s.sale_date) AS sale_time,
       COALESCE(c.customer_name, 'Walk-in Customer') AS customer,
       s.payment_method, s.total_amount
FROM sales s
LEFT JOIN customers c ON c.id = s.customer_id
WHERE DATE(s.sale_date) = CURDATE()
ORDER BY s.sale_date DESC;
```

Output: 4 rows

invoice_no	sale_time	customer	payment_method	total_amount
INV-000051	15:41:07	Aarav Mehta	Card	2239.64
INV-000050	15:11:07	Neha Patil	UPI	585.28
INV-000049	14:26:07	Kavya Joshi	Other	377.60
INV-000048	13:31:07	Meera Pillai	UPI	2272.68

10.6 Top 10 selling products (by quantity)

Concepts: JOIN, GROUP BY, SUM, ORDERBY, LIMIT

Units sold and revenue per product, ranked by quantity sold.

```
SELECT p.product_name, SUM(si.quantity) AS total_sold, SUM(si.subtotal) AS revenue
FROM sale_items si
JOIN products p ON si.product_id = p.id
GROUP BY p.id, p.product_name
ORDER BY total_sold DESC
LIMIT 10;
```

Output: 10 rows (first 6 shown)

product_name	total_sold	revenue
Fluoride Toothpaste 150g	26	2574.00
Glucose Biscuits 800g	21	1995.00
Floor Cleaner 1L	17	3383.00
LED Bulb 9W (Pack of 2)	16	3184.00
Optical USB Mouse	16	5584.00
Neem Bath Soap (Pack of 4)	16	2560.00

10.7 Monthly sales

Concepts: GROUPBY, DATE_FORMAT, COUNT, SUM

Revenue, order count and average order value grouped by calendar month.

```
SELECT DATE_FORMAT(sale_date, '%Y-%m') AS month,
       COUNT(*) AS orders,
       SUM(total_amount) AS revenue,
       ROUND(AVG(total_amount), 2) AS avg_order_value
FROM sales
GROUP BY DATE_FORMAT(sale_date, '%Y-%m')
ORDER BY month;
```

Output: 4 rows

month	orders	revenue	avg_order_value
2026-07	7	13654.96	1950.71
2026-08	11	18184.98	1653.18
2026-09	24	34867.82	1452.83
2026-10	9	13624.28	1513.81

10.8 Total sales by category

Concepts: Multi-table JOIN, GROUPBY, SUM

Joins sale_items → products → categories to calculate revenue and units sold per category.

```
SELECT c.category_name, SUM(si.quantity) AS units_sold, SUM(si.subtotal) AS revenue
FROM sale_items si
JOIN products p ON si.product_id = p.product_id
JOIN categories c ON c.id = p.category_id
GROUP BY c.id, c.category_name
ORDER BY revenue DESC;
```

Output: 12 rows (first 6 shown)

category_name	units_sold	revenue
Computer Accessories	20	9980.00
Mobile Accessories	18	9832.00
Groceries	41	9381.00
Home Appliances	21	8929.00
Personal Care	48	6604.00
Cleaning Products	27	5176.00

10.9 Customers with the highest purchase amounts

Concepts: JOIN, GROUP BY, COUNT, SUM, ORDERBY

Total spend and number of orders for each customer, highest spenders first.

```
SELECT c.customer_name, c.phone, COUNT(s.id) AS orders, SUM(s.total_amount) AS total_spent,
       MAX(s.sale_date) AS last_purchase
FROM customers c
JOIN sales s ON s.customer_id = c.id
GROUP BY c.id, c.customer_name, c.phone
ORDER BY total_spent DESC
LIMIT 10;
```

Output: 10 rows (first 6 shown)

customer_name	phone	orders	total_spent	last_purchase
Lakshmi Krishnan	9894154091	3	7773.84	2026-09-30 18:03:00
Meera Pillai	9872722589	4	7638.14	2026-10-05 13:31:07
Shreya Ghosh	9887010257	4	6730.72	2026-09-28 12:13:00
Vikram Singh	9826287668	3	6045.14	2026-09-26 10:05:00
Rahul Gupta	9840575336	3	5216.78	2026-09-28 18:52:00
Aarav Mehta	9812000000	3	4065.10	2026-10-05 15:41:07

10.10 Total inventory value (with ROLLUP)

Concepts: SUM, GROUP BY ... WITH ROLLUP, GROUPING()

Stock value at cost price per category. WITH ROLLUP adds a grand-total row; GROUPING() labels it.

```
SELECT IF(GROUPING(c.category_name), 'ALL CATEGORIES (TOTAL)', c.category_name) AS category,
       SUM(i.quantity) AS units_in_stock,
       SUM(i.quantity * p.purchase_price) AS value_at_cost,
       SUM(i.quantity * p.selling_price) AS value_at_retail
FROM products p
JOIN inventory i ON i.product_id = p.id
JOIN categories c ON c.id = p.category_id
GROUP BY c.category_name WITH ROLLUP;
```

Output: 13 rows (first 6 shown)

category	units_in_stock	value_at_cost	value_at_retail
Beverages	140	19968.00	26590.00
Cleaning Products	74	12710.00	16970.00
Computer Accessories	18	9540.00	15282.00
Dairy & Bakery	16	3920.00	4560.00
Electronics	15	78900.00	106185.00
Groceries	282	44124.00	56866.00

10.11 Products that have never been sold

Concepts: Subquery, NOT EXISTS

A correlated subquery finds products with no matching row in sale_items.

```
SELECT p.id, p.product_name, p.sku, p.selling_price
FROM products p
WHERE NOT EXISTS (
    SELECT 1 FROM sale_items si WHERE si.product_id = p.id
);
```

Output: 2 rows

id	product_name	sku	selling_price
27	Smart LED TV 32"	ELC-TV-32IN	14999.00
31	Non-stick Frying Pan 24cm	KIT-FPAN-24	799.00

10.12 Best-selling products (HAVING filter)

Concepts: GROUP BY, HAVING, SUM, Profit calculation

Products that sold at least 8 units, with gross profit = revenue – cost. HAVING filters groups after aggregation (WHERE cannot).

```
SELECT p.product_name,
       SUM(si.quantity) AS units_sold,
       SUM(si.subtotal) AS revenue,
       SUM(si.subtotal) - SUM(si.quantity * p.purchase_price) AS gross_profit
FROM sale_items si
JOIN products p ON p.id = si.product_id
GROUP BY p.id, p.product_name
HAVING SUM(si.quantity) >= 8
ORDER BY gross_profit DESC;
```

Output: 15 rows (first 6 shown)

product_name	units_sold	revenue	gross_profit
Optical USB Mouse	16	5584.00	2224.00
Steel Water Bottle 1L	10	3990.00	1590.00
USB Type-C Cable 1m	9	2241.00	1161.00
LED Bulb 9W (Pack of 2)	16	3184.00	944.00
Floor Cleaner 1L	17	3383.00	918.00
Fluoride Toothpaste 150g	26	2574.00	806.00

10.13 Sales with customer and cashier details

Concepts: LEFT JOIN, INNER JOIN, Correlated subquery

Each invoice with its customer (LEFT JOIN for walk-ins), the cashier who billed it, and its item count from a correlated subquery.

```
SELECT s.invoice_no, s.sale_date,
       COALESCE(c.customer_name, 'Walk-in Customer') AS customer,
       u.name AS billed_by,
       (SELECT COUNT(*) FROM sale_items si WHERE si.sale_id = s.id) AS items,
       s.total_amount, s.payment_method, s.payment_status
FROM sales s
LEFT JOIN customers c ON c.id = s.customer_id
INNER JOIN users u ON u.id = s.user_id
ORDER BY s.sale_date DESC
LIMIT 25;
```

Output: 25 rows (first 6 shown)

invoice_no	sale_date	customer	billed_by	items	total_amount	payment_method	payment_status
INV-000051	2026-10-05 15:41:07	Aarav Mehta	Rahul Verma	4	2239.64	Card	Paid
INV-000050	2026-10-05 15:11:07	Neha Patil	Rahul Verma	2	585.28	UPI	Paid
INV-000049	2026-10-05 14:26:07	Kavya Joshi	Admin User	1	377.60	Other	Pending
INV-000048	2026-10-05 13:31:07	Meera Pillai	Rahul Verma	4	2272.68	UPI	Paid
INV-000047	2026-10-04 20:18:00	Ananya Iyer	Priya Sharma	4	2266.78	UPI	Paid
INV-000046	2026-10-04 16:12:00	Divya Menon	Admin User	2	947.54	UPI	Paid

10.14 Total quantity sold per product (including zero)

Concepts: LEFT JOIN, COALESCE, GROUP BY

LEFT JOIN from products keeps unsold products in the result with 0 units.

```
SELECT p.product_name, COALESCE(SUM(si.quantity), 0) AS total_quantity_sold
FROM products p
LEFT JOIN sale_items si ON si.product_id = p.id
GROUP BY p.id, p.product_name
ORDER BY total_quantity_sold DESC;
```

Output: 32 rows (first 6 shown)

product_name	total_quantity_sold
Fluoride Toothpaste 150g	26
Glucose Biscuits 800g	21
Floor Cleaner 1L	17
LED Bulb 9W (Pack of 2)	16
Neem Bath Soap (Pack of 4)	16
Optical USB Mouse	16

10.15 Suppliers supplying the most products

Concepts: LEFT JOIN, GROUP BY, COUNT, SUM

Number of products and total stock value per supplier.

```
SELECT s.supplier_name, s.contact_person,
       COUNT(p.id) AS products_supplied,
       COALESCE(SUM(i.quantity), 0) AS units_in_stock,
       COALESCE(SUM(i.quantity * p.purchase_price), 0) AS stock_value
FROM suppliers s
LEFT JOIN products p ON p.supplier_id = s.id
LEFT JOIN inventory i ON i.product_id = p.id
GROUP BY s.id, s.supplier_name, s.contact_person
ORDER BY products_supplied DESC, stock_value DESC;
```

Output: 8 rows (first 6 shown)

supplier_name	contact_person	products_supplied	units_in_stock	stock_value
Shree Ganesh Traders	Mahesh Patil	6	204	33297.00
Metro FMCG Wholesale	Sandeep Kulkarni	6	295	17008.00
Annapurna Foods Pvt Ltd	Sunita Joshi	5	138	34354.00
TechNova Distributors	Kiran Rao	4	33	88440.00
Orbit Mobile Accessories	Imran Shaikh	3	101	24630.00
Bharat Electricals	Jignesh Shah	3	104	22740.00

10.16 Customers who spend more than the average customer

Concepts: Nested subquery, Derived table, HAVING

The subquery computes the average total spend per customer; HAVING keeps only customers above it.

```
SELECT c.customer_name, SUM(s.total_amount) AS total_spent
FROM customers c
JOIN sales s ON s.customer_id = c.id
GROUP BY c.id, c.customer_name
HAVING SUM(s.total_amount) > (
    SELECT AVG(customer_total)
    FROM (SELECT SUM(total_amount) AS customer_total
          FROM sales
          WHERE customer_id IS NOT NULL
          GROUP BY customer_id) AS per_customer
)
ORDER BY total_spent DESC;
```

Output: 9 rows (first 6 shown)

customer_name	total_spent
Lakshmi Krishnan	7773.84
Meera Pillai	7638.14
Shreya Ghosh	6730.72
Vikram Singh	6045.14
Rahul Gupta	5216.78
Aarav Mehta	4065.10

10.17 Payment method & status summary

Concepts: GROUPBY multiple columns, COUNT, SUM

How many payments, and how much money, per payment method and status.

```
SELECT payment_method, payment_status, COUNT(*) AS payments, SUM(amount) AS amount
FROM payments
GROUP BY payment_method, payment_status
ORDER BY payment_method, payment_status;
```

Output: 6 rows

payment_method	payment_status	payments	amount
Cash	Paid	14	18290.00
UPI	Paid	23	40267.50
Card	Paid	7	13376.48
Card	Failed	3	3387.78
Other	Paid	4	5558.98
Other	Pending	3	2839.08

10.18 Daily sales for the last 7 days

Concepts: Date filtering, BETWEEN, DATE_SUB, GROUP BY

Uses BETWEEN with DATE_SUB(CURDATE(), INTERVAL 6 DAY) to filter a rolling 7-day window.

```
SELECT DATE(sale_date) AS sale_day, COUNT(*) AS orders, SUM(total_amount) AS revenue
FROM sales
WHERE DATE(sale_date) BETWEEN DATE_SUB(CURDATE(), INTERVAL 6 DAY) AND CURDATE()
GROUP BY DATE(sale_date)
ORDER BY sale_day;
```

Output: 5 rows

sale_day	orders	revenue
2026-09-29	1	2397.76
2026-09-30	1	1154.04
2026-10-03	2	4701.12
2026-10-04	3	3447.96
2026-10-05	4	5475.20

10.19 Querying a VIEW (v_product_stock)

Concepts: VIEW, CASE, Derived columns

v_product_stock is a stored view joining 4 tables. It is queried like a normal table.

```
SELECT product_name, category_name, supplier_name, stock, reorder_level, stock_status, inventory_value
FROM v_product_stock
WHERE stock_status <> 'In Stock'
ORDER BY stock;
```

Output: 8 rows (first 6 shown)

product_name	category_name	supplier_name	stock	reorder_level	stock_status	inventory_value
Masala Potato Chips 90g	Snacks	Metro FMCG Wholesale	0	30	Out of Stock	0.00
Dishwash Gel 750ml	Cleaning Products	Sparkle Home Care	0	15	Out of Stock	0.00
Non-stick Frying Pan 24cm	Kitchen Items	Bharat Electricals	3	5	Low Stock	1560.00
Optical USB Mouse	Computer Accessories	TechNova Distributors	6	10	Low Stock	1260.00
Gel Pen Set (10 pcs)	Stationery	Shree Ganesh Traders	7	20	Low Stock	455.00
Fluoride Toothpaste 150g	Personal Care	Shree Ganesh Traders	9	25	Low Stock	612.00

10.20 Categories with product counts (including empty)

Concepts: LEFT JOIN, GROUP BY, COUNT

COUNT(p.id) with a LEFT JOIN returns 0 for categories that have no products yet.

```
SELECT c.id, c.category_name, COUNT(p.id) AS product_count
FROM categories c
LEFT JOIN products p ON p.category_id = c.id
GROUP BY c.id, c.category_name
ORDER BY product_count DESC, c.category_name;
```

Output: 13 rows (first 6 shown)

category_name	product_count
Groceries 1	5
Beverages 3	4
Cleaning Products 5	3
Mobile Accessories 7	3
Personal Care 4	3
Snacks 2	3

10.21 Rank products by revenue within each category

Concepts: Window function, RANK()OVER(PARTITION BY)

A MySQL 8 window function ranks each product inside its own category without collapsing rows.

```
SELECT category_name, product_name, revenue,
       RANK() OVER (PARTITION BY category_name ORDER BY revenue DESC) AS rank_in_category
FROM (
  SELECT c.category_name, p.product_name, COALESCE(SUM(si.subtotal), 0) AS revenue
  FROM products p
  JOIN categories c ON c.id = p.category_id
  LEFT JOIN sale_items si ON si.product_id = p.id
  GROUP BY c.category_name, p.id, p.product_name
) AS product_revenue
ORDER BY category_name, rank_in_category;
```

Output: 32 rows (first 6 shown)

category_name	product_name	revenue	rank_in_category
Beverages	Assam Tea 500g	1100.00	1
Beverages	Cola Soft Drink 2L	1045.00	2
Beverages	Instant Coffee 100g	680.00	3
Beverages	Mango Fruit Drink 1.2L	300.00	4
Cleaning Products	Floor Cleaner 1L	3383.00	1
Cleaning Products	Dishwash Gel 750ml	1043.00	2

10.22 Stock movement audit trail

Concepts: JOIN, LEFTJOIN, ORDER BY,LIMIT

The most recent inventory changes (sales, restocks, adjustments) with who made them.

```
SELECT m.created_at, p.product_name, m.movement_type, m.quantity_change, m.quantity_after,
       COALESCE(u.name, 'System') AS performed_by, m.reason
FROM stock_movements m
JOIN products p ON p.id = m.product_id
LEFT JOIN users u ON u.id = m.user_id
ORDER BY m.created_at DESC, m.id DESC
LIMIT 25;
```

Output: 25 rows (first 6 shown)

created_at	product_name	movement_type	quantity_change	quantity_after	performed_by	reason
2026-10-05 15:41:07	Iodized Salt 1kg	SALE	-3	120	Rahul Verma	Sold on INV-000051
2026-10-05 15:41:07	Neem Bath Soap (Pack of 4)	SALE	-3	55	Rahul Verma	Sold on INV-000051
2026-10-05 15:41:07	Toor Dal 1kg	SALE	-4	64	Rahul Verma	Sold on INV-000051
2026-10-05 15:41:07	Optical USB Mouse	SALE	-2	6	Rahul Verma	Sold on INV-000051
2026-10-05 15:11:07	Fluoride Toothpaste 150g	SALE	-3	9	Rahul Verma	Sold on INV-000050
2026-10-05 15:11:07	LED Bulb 9W (Pack of 2)	SALE	-1	90	Rahul Verma	Sold on INV-000050

11. UI Design and Screenshots

11.1 Design approach

The interface is a single-page React application with a fixed left navigation (grouped as Overview, Catalog, Operations and Insights), a top bar with global search, today's date, a notification bell (low stock and pending payments) and the signed-in user. The colour scheme is a calm indigo on white with semantic colours: **green** for in stock / paid, **amber** for low stock / pending and **red** for out of stock / failed. The layout is responsive (the sidebar becomes a drawer on phones), forms are validated on the client and the server, and every list has search, filters, sorting and pagination. Loading skeletons, empty states, toasts and confirmation dialogs give feedback for every action.

11.2 Screenshots

All screenshots were captured from the running application connected to the seeded MySQL database (admin session, 5 October 2026).

11.2.1 Login

Staff sign in with e-mail and password. The server verifies the bcrypt hash and returns a signed JWT that is attached to every later request. The page offers one-click buttons for the three demo roles.

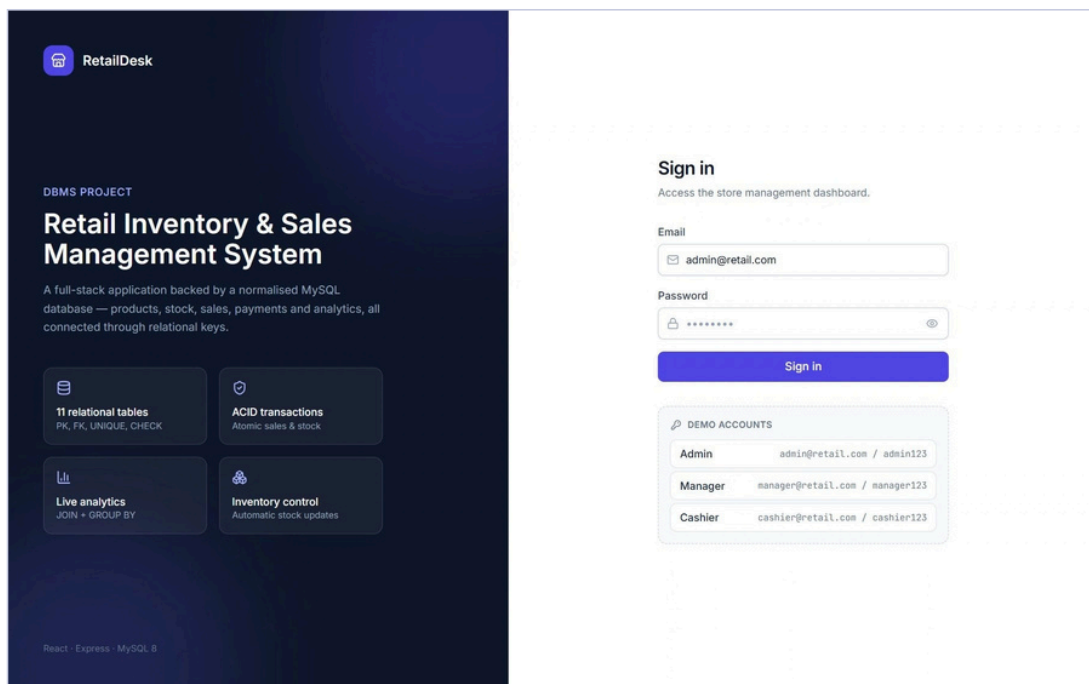


Figure 2: Login page with one-click demo accounts

11.2.2 Dashboard

Eight KPI cards, a daily / weekly / monthly sales chart, sales by category, top-selling products, inventory status, best customers, recent invoices and low-stock alerts. Every value is calculated by SQL at the moment the page is opened.

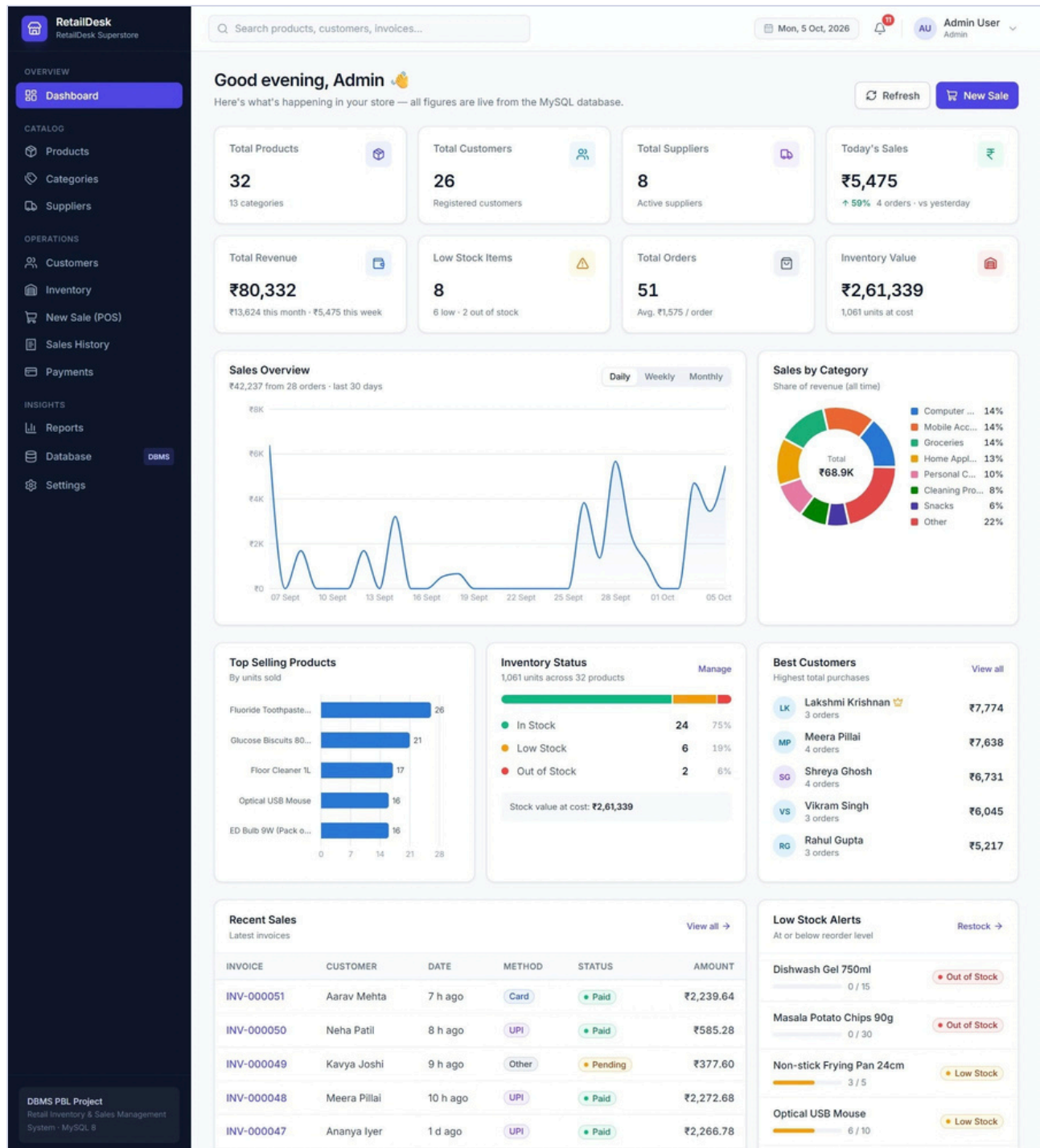


Figure 3: Dashboard with KPI cards, sales trend, category share, top sellers, inventory status, best customers, recent sales and low-stock alerts

11.2.3 Products

FullCRUD with search, category / supplier / status filters, sortable columns and pagination. Validation rejects a duplicate SKU or a negative price. The view modal shows sales statistics and stock history.

ID	PRODUCT	SKU	CATEGORY	SUPPLIER	PURCHASE	SELLING	STOCK	REORDER
#1	Premium Basmati Rice 5kg	GRS-RICE-5KG	Groceries	Kaveri Agro Suppliers	₹420.00	₹549.00	48	15
#2	Toor Dal 1kg	GRS-TOOR-1KG	Groceries	Kaveri Agro Suppliers	₹125.00	₹159.00	64	20
#3	Refined Sunflower Oil 1L	GRS-OIL-1L	Groceries	Annapurna Foods Pvt Ltd	₹118.00	₹145.00	18	25
#4	Whole Wheat Atta 10kg	GRS-ATTA-10KG	Groceries	Annapurna Foods Pvt Ltd	₹365.00	₹449.00	32	10
#5	Iodized Salt 1kg	GRS-SALT-1KG	Groceries	Metro FMCG Wholesale	₹18.00	₹28.00	120	30
#6	Glucose Biscuits 800g	SNK-BISC-800g	Snacks	Metro FMCG Wholesale	₹72.00	₹95.00	85	20
#7	Masala Potato Chips 90g	SNK-CHIP-90g	Snacks	Metro FMCG Wholesale	₹28.00	₹40.00	0	30
#8	Salted Cashews 250g	SNK-CASH-250g	Snacks	Annapurna Foods Pvt Ltd	₹245.00	₹320.00	22	10
#9	Cola Soft Drink 2L	BEV-COLA-2L	Beverages	Metro FMCG Wholesale	₹68.00	₹95.00	60	24
#10	Mango Fruit Drink 1.2L	BEV-MANGO-1L	Beverages	Metro FMCG Wholesale	₹52.00	₹75.00	14	20

Figure 4: Products list with filters, status badges and actions

11.2.4 Inventory

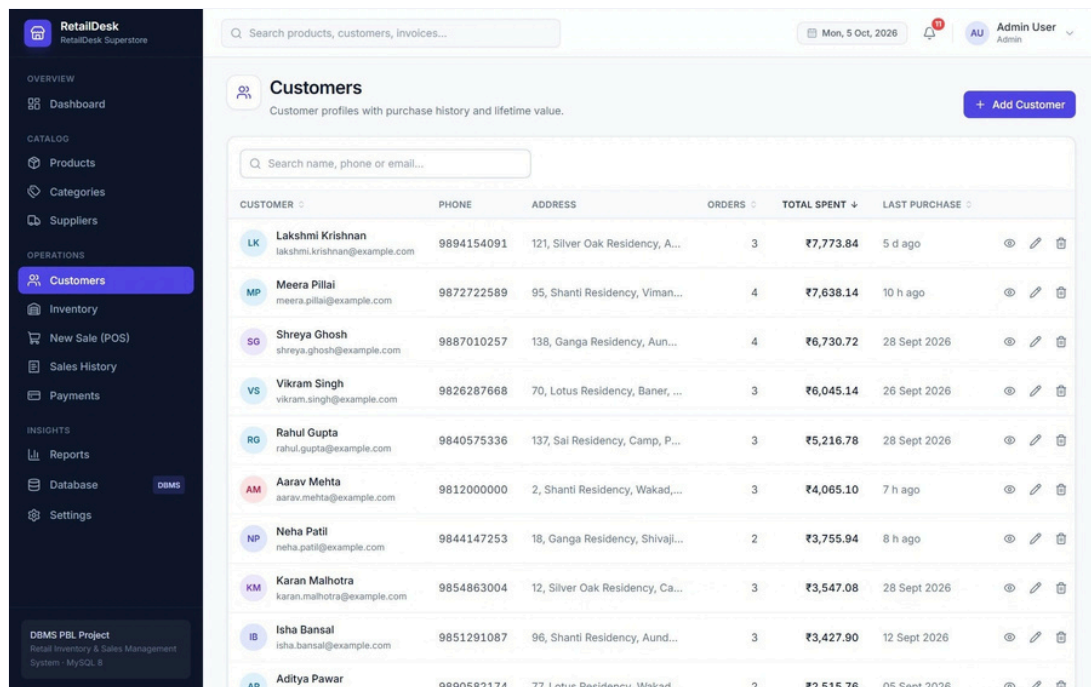
Livestock levels from the `v_product_stock` view. Stock can be adjusted (in, out or set); every change is recorded in the `stock_movements` audit log.

PRODUCT	CATEGORY	CURRENT STOCK	REORDER LEVEL	STATUS	INVENTORY VALUE
Masala Potato Chips 90g SNK-CHIP-90g	Snacks	0	30	Out of Stock	₹0.00
Dishwash Gel 750ml CLN-DISH-750	Cleaning Products	0	15	Out of Stock	₹0.00
Non-stick Frying Pan 24cm KIT-FPAN-24	Kitchen Items	3	5	Low Stock	₹1,560.00
Optical USB Mouse CMP-MOUSE-USB	Computer Accessories	6	10	Low Stock	₹1,260.00
Smart LED TV 32" ELC-TV-32IN	Electronics	6	3	In Stock	₹69,000.00
Gel Pen Set (10 pcs) STR-PEN-10PK	Stationery	7	20	Low Stock	₹455.00
Fluoride Toothpaste 150g PC-TPST-150	Personal Care	9	25	Low Stock	₹612.00
Bluetooth Speaker 10W	Electronics	9	5	In Stock	₹9,900.00

Figure 5: Inventory with In Stock / Low Stock / Out of Stock badges

11.2.5 Customers

Customer master data with the number of orders and total spent computed by JOIN and GROUP BY. The detail view lists purchase history and favourite products.

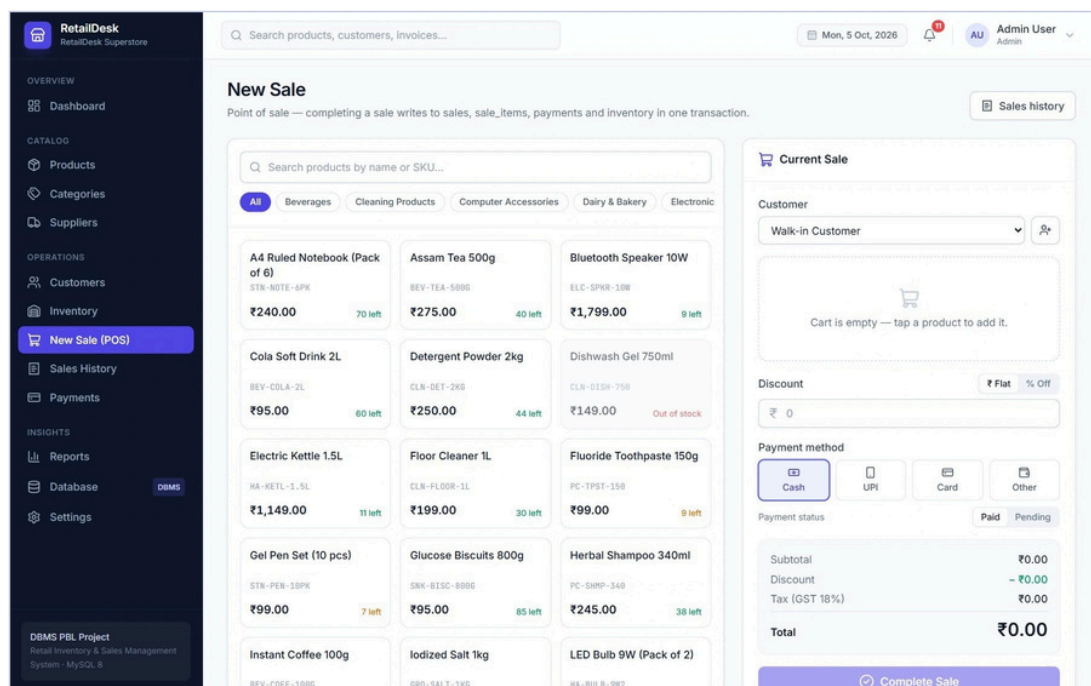


CUSTOMER	PHONE	ADDRESS	ORDERS	TOTAL SPENT	LAST PURCHASE
LK Lakshmi Krishnan lakshmi.krishnan@example.com	9894154091	121, Silver Oak Residency, A...	3	₹7,773.84	5 d ago
MP Meera Pillai meera.pillai@example.com	9872722589	95, Shanti Residency, Viman...	4	₹6,638.14	10 h ago
SG Shreya Ghosh shreya.ghosh@example.com	9887010257	138, Ganga Residency, Aun...	4	₹6,730.72	28 Sept 2026
VS Vikram Singh vikram.singh@example.com	9826287668	70, Lotus Residency, Baner, ...	3	₹6,045.14	26 Sept 2026
RG Rahul Gupta rahul.gupta@example.com	9840575336	137, Sai Residency, Camp, P...	3	₹5,216.78	28 Sept 2026
AM Aarav Mehta aarav.mehta@example.com	9812000000	2, Shanti Residency, Wakad, ...	3	₹4,065.10	7 h ago
NP Neha Patil neha.patil@example.com	9844147253	18, Ganga Residency, Shivaji...	2	₹3,755.94	8 h ago
KM Karan Malhotra karan.malhotra@example.com	9854863004	12, Silver Oak Residency, Ca...	3	₹3,547.08	28 Sept 2026
IB Isha Bansal isha.bansal@example.com	9851291087	96, Shanti Residency, Aund...	3	₹3,427.90	12 Sept 2026
AP Aditya Pawar aditya.pawar@example.com	9890582174	77, Lotus Residency, Wakad, ...	2	₹2,515.76	05 Sept 2026

Figure 6: Customers with orders and total spent

11.2.6 New Sale (Point of Sale)

The cashier picks a customer (or walk-in), searches or taps products, changes quantities, applies a flat or percentage discount, chooses the payment method and completes the sale. GST is calculated automatically and the sale is saved in one database transaction.



Product	Price	Stock
A4 Ruled Notebook (Pack of 6)	₹240.00	70 left
Assam Tea 500g	₹275.00	40 left
Bluetooth Speaker 10W	₹1,799.00	9 left
Cola Soft Drink 2L	₹95.00	60 left
Detergent Powder 2kg	₹250.00	44 left
Dishwash Gel 750ml	₹149.00	Out of stock
Electric Kettle 1.5L	₹1,149.00	11 left
Floor Cleaner 1L	₹199.00	30 left
Fluoride Toothpaste 150g	₹99.00	9 left
Gel Pen Set (10 pcs)	₹99.00	7 left
Glucose Biscuits 800g	₹95.00	85 left
Herbal Shampoo 340ml	₹245.00	38 left
Instant Coffee 100g	₹99.00	100 left
Iodized Salt 1kg	₹99.00	100 left
LED Bulb 9W (Pack of 2)	₹99.00	100 left

Current Sale

Customer: Walk-in Customer

Search products by name or SKU...

Discount: ₹ Flat % Off

Payment method: Cash UPI Card Other

Payment status: Paid Pending

Subtotal: ₹0.00

Discount: -₹0.00

Tax (GST 18%): ₹0.00

Total: ₹0.00

Complete Sale

Figure 7: Point-of-sale screen

11.2.7 Sales History

All invoices with search, date-range, payment-method and status filters, a detail modal, CSV export and a link to the printable invoice.

Sales History
Every completed sale with its items, totals and payment status.

Summary Statistics:

- Sales: 51
- Revenue: ₹80,332
- Average sale: ₹1,575
- Discounts given: ₹783

Filters:

- Search Invoice, customer or phone:
- Date Range: dd-mm-yyyy to dd-mm-yyyy
- Methods: All methods
- Status: All status
- Time Period: Today, 7 days, 30 days

SALE ID	DATE	CUSTOMER	ITEMS	SUBTOTAL	DISCOUNT	TAX	TOTAL	METHOD	STATUS
INV-000051	05 Oct 2026, 03:41 pm	Aarav Mehta	12	₹1,898.00	—	₹341.64	₹2,239.64	Card	Paid
INV-000050	05 Oct 2026, 03:11 pm	Neha Patil	4	₹496.00	—	₹69.28	₹565.28	UPI	Paid
INV-000049	05 Oct 2026, 02:26 pm	Kavya Joshi	2	₹320.00	—	₹57.60	₹377.60	Other	Pending
INV-000048	05 Oct 2026, 01:31 pm	Meera Pillai	5	₹2,027.00	-₹101.00	₹346.68	₹2,272.68	UPI	Paid
INV-000047	04 Oct 2026, 08:18 pm	Ananya Iyer	8	₹1,921.00	—	₹345.78	₹2,266.78	UPI	Paid
INV-000046	04 Oct 2026, 04:12 pm	Divya Menon	5	₹845.00	-₹42.00	₹144.54	₹947.54	UPI	Paid
INV-000045	04 Oct 2026, 01:23 pm	Sneha Kulkarni	2	₹198.00	—	₹35.64	₹233.64	Cash	Paid
INV-000044	03 Oct 2026, 06:15 pm	Meera Pillai	5	₹3,336.00	—	₹600.48	₹3,936.48	Cash	Paid
INV-000043	03 Oct 2026, 04:35 pm	Meera Pillai	2	₹648.00	—	₹116.64	₹764.64	Other	Pending

Figure 8: Sales history with filters

11.2.8 Tax Invoice

A professional tax invoice with store details, GSTIN, customer, line items, subtotal, discount, GST and grand total. *Print → Save as PDF* produces a PDF copy.

 **RetailDesk**
RetailDesk Superstore

OVERVIEW

Dashboard

CATALOG

Products

Categories

Suppliers

OPERATIONS

Customers

Inventory

New Sale (POS)

Sales History

Payments

INSIGHTS

Reports

Database

Settings

DBMS

Search products, customers, invoices...

11

AU

Admin User

Admin

← Back to sales

Print Invoice

 **RetailDesk Superstore**
FC Road, Shivajinagar, Pune, Maharashtra
411005
9876543210 · contact@retaildesk.example.com
GSTIN: 27ABCDE1234F1Z5

TAX INVOICE
Invoice No: **INV-000051**
Date: 05 Oct 2026, 03:41 pm
Billed by: Rahul Verma

BILL TO
Aarav Mehta
9812000000
aarav.mehta@example.com
2, Shanti Residency, Wakad, Pune 411057

PAYMENT
Method: **Card**
Status: **Paid**
Ref: CRD646347

#	ITEM	QTY	UNIT PRICE	AMOUNT
1	Optical USB Mouse CMP-MOUS-USB	2	₹349.00	₹698.00
2	Toor Dal 1kg GRD-TOOR-1KG	4	₹159.00	₹636.00
3	Neem Bath Soap (Pack of 4) PC-SOAP-4PK	3	₹160.00	₹480.00
4	Iodized Salt 1kg GRD-SALT-1KG	3	₹28.00	₹84.00
Subtotal				₹1,898.00
Discount				- ₹0.00
Tax (GST 18%)				₹341.64
Grand Total				₹2,239.64

Thank you for shopping with us!
Goods once sold can be exchanged within 7 days with this invoice.

Authorized Signatory

Figure 9: Print-ready tax invoice

11.2.9 Payments

Payment records with status filters. A pending payment can be marked as paid or failed; the related sale is updated in the same transaction.

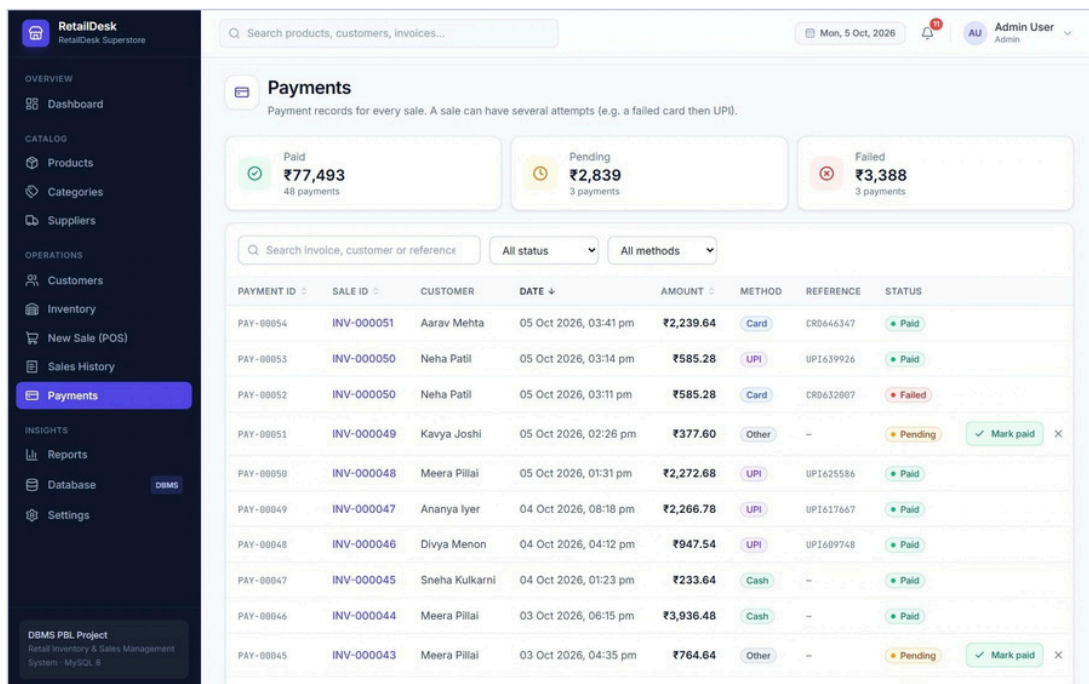


Figure 10: Payments with Paid / Pending / Failed badges

11.2.10 Reports

Ninereports (daily, weekly, monthly, product, category, customer, inventory, low stock and supplier) with date filters, chart, totals row, CSV export and print.

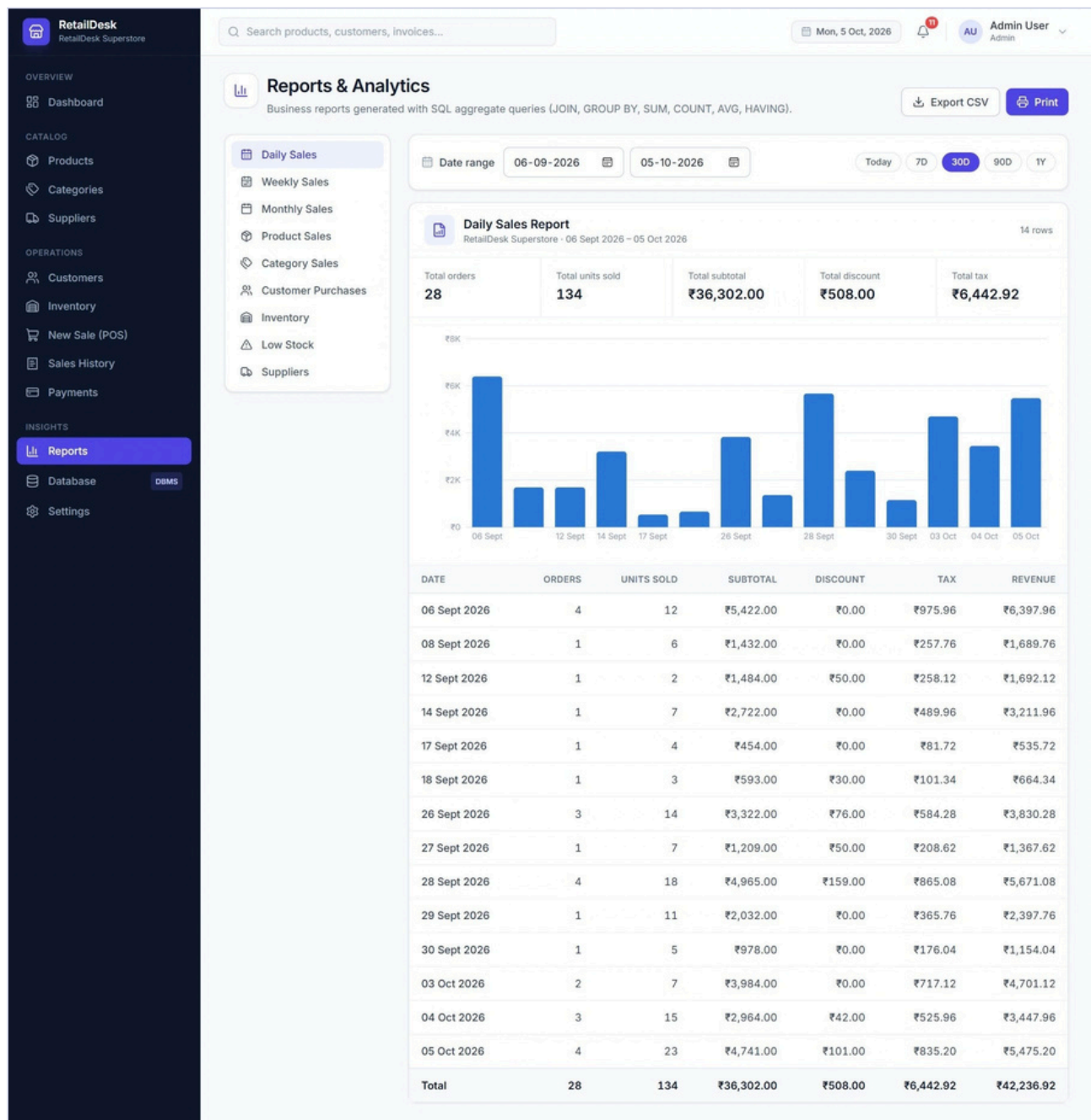


Figure 11: Daily sales report with chart, totals row and CSV / Print actions

11.2.11 Database page: overview

The Database page presents the DBMS side of the project: summary counters, relationships, normalisation and the list of DBMS concepts demonstrated.

RetailDesk
RetailDesk Superstore

OVERVIEW

- Dashboard

CATALOG

- Products
- Categories
- Suppliers

OPERATIONS

- Customers
- Inventory
- New Sale (POS)
- Sales History
- Payments

INSIGHTS

- Reports
- Database** (DBMS)
- Settings

Search products, customers, invoices...

Mon, 5 Oct, 2026

Admin User Admin

Database

Live view of the MySQL schema, relationships and SQL behind this application.

retail_inventory_db - MySQL 8.0.46

Overview ER Diagram Table Schemas SQL Queries Transactions

11 Tables

539 Total rows

11 Foreign keys

13 CHECK constraints

83 Columns

1 Views

DBMS Concepts Used

Concepts demonstrated by this project

- Relational Database**
11 related tables in MySQL 8 (InnoDB engine).
- Foreign Keys**
11 FK constraints enforce every relationship.
- CRUD Operations**
INSERT / SELECT / UPDATE / DELETE from every module.
- Joins**
INNER JOIN and LEFT JOIN across up to 4 tables.
- Transactions**
Sales & stock changes run in START TRANSACTION ... COMMIT / ROLLBACK.
- Database Relationships**
1:1 (product-inventory) and 1:N relationships.
- Views**
v_product_stock joins 4 tables with a derived stock status.
- Subqueries**
Correlated, NOT EXISTS and derived-table subqueries.
- Primary Keys**
Every table has an AUTO_INCREMENT surrogate key 'id'.
- Normalization**
Schema in 3NF — no repeating groups, partial or transitive dependencies.
- SQL Queries**
22 demonstration queries + every screen is SQL-backed.
- Aggregate Functions**
COUNT, SUM, AVG, MIN, MAX with GROUP BY / HAVING.
- Referential Integrity**
ON DELETE RESTRICT / CASCADE / SET NULL chosen per relationship.
- Constraints**
NOT NULL, UNIQUE, CHECK (e.g. stock ≥ 0, price ≥ 0).
- Indexes**
Indexes on FKs, dates and names speed up joins and filters.
- Generated Columns**
sale_items.subtotal = quantity × unit_price (STORED).

Relationships

Read live from INFORMATION_SCHEMA

- products 1 — 1 inventory**
via product_id · ON DELETE CASCADE
- sales 1 — N payments**
via sale_id · ON DELETE CASCADE
- categories 1 — N products**
via category_id · ON DELETE RESTRICT
- suppliers 1 — N products**
via supplier_id · ON DELETE SET NULL
- products 1 — N sale_items**
via product_id · ON DELETE RESTRICT
- sales 1 — N sale_items**
via sale_id · ON DELETE CASCADE
- customers 1 — N sales**
via customer_id · ON DELETE SET NULL
- users 1 — N sales**
via user_id · ON DELETE RESTRICT
- products 1 — N stock_movements**
via product_id · ON DELETE CASCADE
- sales 1 — N stock_movements**
via sale_id · ON DELETE SET NULL
- users 1 — N stock_movements**
via user_id · ON DELETE SET NULL

Normalization

Why the design is in Third Normal Form

- 1NF** Atomic values only — a sale's products live in separate sale_items rows, not a comma list.
- 2NF** No partial dependency — product details depend on products.id, not on a (sale_id, product_id) key.
- 3NF** No transitive dependency — category and supplier names are stored once and referenced by FK.

Figure 12: Database overview

11.2.12 Database page: ER diagram

AliveERdiagramwithPK/FK/UQmarkersandrow counts. Hovering over a table highlights its relationships.

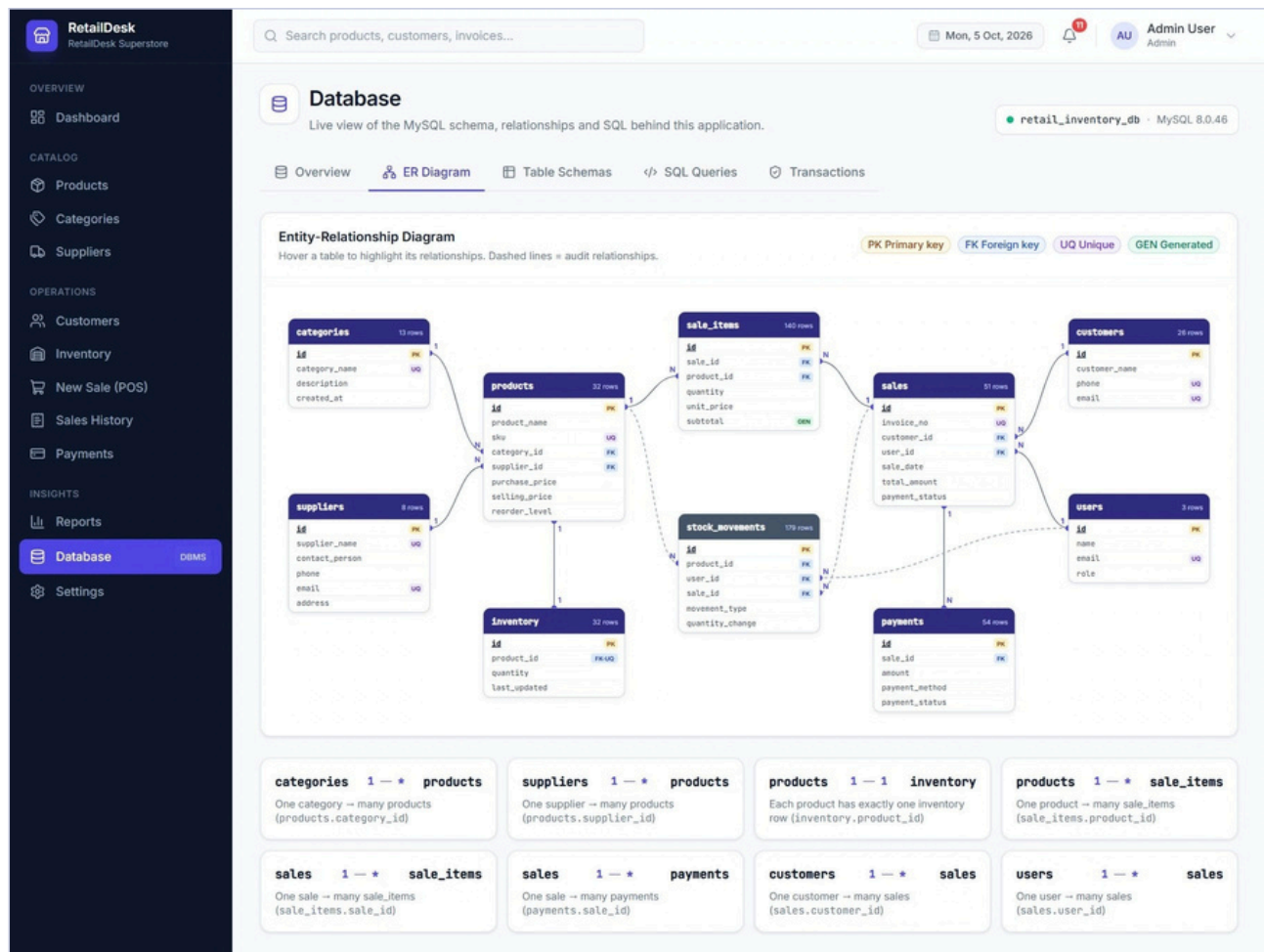


Figure 13: Live ER diagram with relationship cards

11.2.13 Database page: table schemas

Columns, datatypes, keys, foreign keys with referential actions, CHECK constraints and indexes of each table, read from INFORMATION_SCHEMA.

Database
Live view of the MySQL schema, relationships and SQL behind this application.

retail_inventory_db - MySQL 8.0.46

Overview ER Diagram **Table Schemas** SQL Queries Transactions

products
11 columns - 32 rows - InnoDB

KEY	COLUMN	DATA TYPE	NULL	DEFAULT / EXTRA
PK	id	int unsigned	NOT NULL	auto_increment
	product_name	varchar(150)	NOT NULL	-
UQ	sku	varchar(50)	NOT NULL	-
FK	category_id	int unsigned	NOT NULL	-
FK	supplier_id	int unsigned	NULL	-
	description	text	NULL	-
	purchase_price	decimal(10,2)	NOT NULL	-
	selling_price	decimal(10,2)	NOT NULL	-
	reorder_level	int	NOT NULL	DEFAULT 10
	created_at	datetime	NOT NULL	DEFAULT CURRENT_TIMESTAMP • DEFAULT_GENERAT...
	updated_at	datetime	NOT NULL	DEFAULT CURRENT_TIMESTAMP • DEFAULT_GENERAT...

Foreign keys (2)

- category_id → categories.id
ON DELETE RESTRICT · ON UPDATE CASCADE
- supplier_id → suppliers.id
ON DELETE SET NULL · ON UPDATE CASCADE

CHECK constraints (3)

- chk_products_purchase_price
(purchase_price ≥ 0)
- chk_products_reorder_level
(reorder_level ≥ 0)
- chk_products_selling_price
(selling_price ≥ 0)

Indexes (5)

- fk_products_category (category_id) INDEX
- fk_products_supplier (supplier_id) INDEX
- idx_products_name (product_name) INDEX
- PRIMARY (id) PRIMARY
- uq_products_sku (sku) UNIQUE

Figure 14: Table schema viewer built from INFORMATION_SCHEMA

11.2.14 Database page: SQL queries

The 22 demonstration queries can be run from the browser. The backend runs only these predefined SELECT statements, by id, inside a READ ONLY transaction.

The screenshot displays the 'Database' page in the RetailDesk application. The left sidebar contains navigation links for Overview, Dashboard, Catalog (Products, Categories, Suppliers), Operations (Customers, Inventory, New Sale (POS), Sales History, Payments), and Insights (Reports, Database, Settings). The main content area is titled 'Database' and shows a live view of the MySQL schema. A search bar at the top allows filtering by title or concept. A list of 14 predefined queries is shown on the left, with query 06, 'Top 10 selling products (by quantity)', selected. The right panel displays the SQL query for query 06, which is a SELECT statement joining products and sale items, grouped by product ID and name, ordered by total sold, and limited to 10 results. Below the query, the 'Query result' table shows 10 rows of data, including product names, total sold, and revenue.

Query 06: Top 10 selling products (by quantity)
 Units sold and revenue per product, ranked by quantity sold.

SQL Query:

```
SELECT p.product_name, SUM(si.quantity) AS total_sold, SUM(si.subtotal) AS revenue
FROM sale_items si
JOIN products p ON si.product_id = p.id
GROUP BY p.id, p.product_name
ORDER BY total_sold DESC
LIMIT 10
```

Query result:

PRODUCT_NAME	TOTAL_SOLD	REVENUE
Fluoride Toothpaste 150g	26	2574
Glucose Biscuits 800g	21	1995
Floor Cleaner 1L	17	3383
LED Bulb 9W (Pack of 2)	16	3184
Optical USB Mouse	16	5584
Neem Bath Soap (Pack of 4)	16	2560
Masala Potato Chips 90g	13	520
Iodized Salt 1kg	12	336
Cola Soft Drink 2L	11	1045

Figure 15: SQL query runner (query 06 executed)

11.2.15 Database page: transactions

The demo inserts a sale and an item, reduces stock, then deliberately tries to sell more than exists. MySQL's CHECK constraint rejects it (error 3819), the transaction is rolled back and the page verifies that no sale was saved and stock is unchanged.

RetailDesk
RetailDesk Superstore

Search products, customers, invoices...

Mon, 5 Oct, 2026 | Admin User

Database

Live view of the MySQL schema, relationships and SQL behind this application.

retail_inventory_db · MySQL 8.0.46

Overview | ER Diagram | Table Schemas | SQL Queries | **Transactions**

Sale transaction (ACID)

What happens when "Complete Sale" is clicked in the POS

```

1  START TRANSACTION;
2
3  -- 1. lock the stock rows (prevents two cashiers overselling)
4  SELECT product_id, quantity FROM inventory
5  WHERE product_id IN (1, 6) FOR UPDATE;
6
7  -- 2. sale header
8  INSERT INTO sales (customer_id, user_id, subtotal, discount,
9  VALUES (1, 1, 739.00, 0.00, 133.02, 872.02, 'Paid', 'UPI');
10 UPDATE sales SET invoice_no = CONCAT('INV-', LPAD(LAST_INSERTED_ID, 10, '0'));
11
12 -- 3. line items (subtotal is a generated column)
13 INSERT INTO sale_items (sale_id, product_id, quantity, unit_price,
14 VALUES (@sale_id, 1, 1, 549.00), (@sale_id, 6, 2, 95.00);
15
16 -- 4. reduce stock - the WHERE guard refuses to oversell
17 UPDATE inventory SET quantity = quantity - 1 WHERE product_id = 1;
18 UPDATE inventory SET quantity = quantity - 2 WHERE product_id = 6;
19

```

Atomicity: All 5 steps or none
Consistency: CHECK stock ≥ 0
Isolation: Row locks (FOR UPDATE)
Durability: InnoDB redo log

Live ROLLBACK demonstration

Starts a sale, then a step fails on purpose Run again

- START TRANSACTION;
Transaction opened
- INSERT INTO sales (...) VALUES (...);
Sale #53 inserted (not yet committed)
- INSERT INTO sale_items (sale_id, product_id, quantity, unit_price) VALUES (53, 5, 1, 28);
1 × Iodized Salt 1kg added to the sale
- UPDATE inventory SET quantity = quantity - 1 WHERE product_id = 5;
Iodized Salt 1kg: stock 120 → 119 (visible only inside this transaction)
- UPDATE inventory SET quantity = quantity - 91 WHERE product_id = 28;
Tried to sell 91 × LED Bulb 9W (Pack of 2) but only 90 in stock → MySQL error 3819: Check constraint 'chk_inventory_quantity' is violated.
- ROLLBACK;
All changes made since START TRANSACTION are undone
- SELECT COUNT(*) FROM sales WHERE id = 53;
Returns 0 — the sale was never saved
- SELECT quantity FROM inventory WHERE product_id = 5;
Iodized Salt 1kg: stock is 120 again (was 120 before the transaction)

Database is consistent — the failed transaction left no trace
Sales before / after: 51 / 51 Iodized Salt 1kg stock: 120 / 120

Figure 16: Live ROLLBACK demonstration

11.2.16 Settings

The Admin maintains store name, contact details, GSTIN, tax rate, low-stock threshold and invoice prefix. The values are stored in the single-row settings table.

RetailDesk
RetailDesk Superstore

OVERVIEW
Dashboard

CATALOG
Products
Categories
Suppliers

OPERATIONS
Customers
Inventory
New Sale (POS)
Sales History
Payments

INSIGHTS
Reports
Database
Settings

DBMS PBL Project
Retail Inventory & Sales Management
System - MySQL 8

Search products, customers, invoices...

Mon, 5 Oct, 2026

Admin User
Admin

Settings

Store configuration, stored in the single-row settings table.

Store information

Store name *
RetailDesk Superstore

Store email
contact@retaildesk.example.com

Store phone
9876543210

Address
FC Road, Shivajinagar, Pune, Maharashtra 411005

GSTIN
27ABCD0E1234F125
Printed on invoices

Your account

Admin User
admin@retail.com
Admin

Database connection

Status
Database
Server

Connected
retail_inventory_db
MySQL 8.0.46

Role permissions

- Admin: Everything, including settings
- Manager: Catalogue, stock, payments, reports
- Cashier: POS sales, customers, view data

Billing & inventory

Currency *
INR — Indian Rupee (₹)

Tax rate (%) *
18
Applied to (subtotal - discount) on every new sale

Low stock threshold *
10
Default reorder level for new products

Invoice prefix *
INV
Next invoice looks like INV-000123

Save settings

Figure 17: Store settings

12. Implementation Details

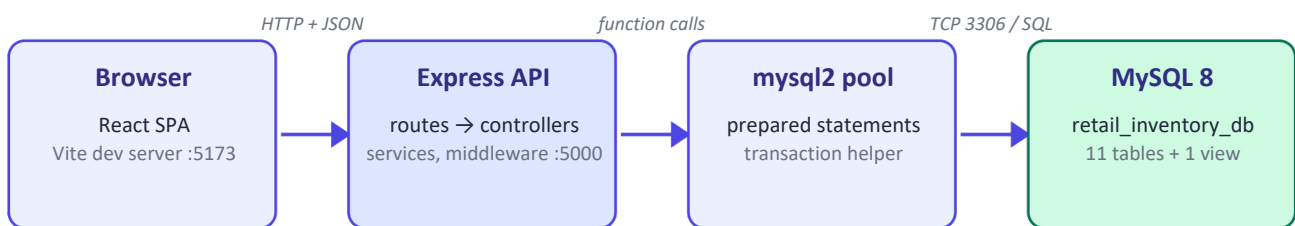
12.1 Technology stack

Table 16: Technology stack

Layer	Technology	Responsibility
Presentation	React 18, React Router 6, Tailwind CSS 3, Recharts, Lucide icons, built with Vite 5	Screens, forms, charts, client-side validation, routing
Application	Node.js 18+, Express 4	REST API, authentication, authorisation, validation, business rules
Data access	mysql2 (connection pool, raw parameterised SQL)	Executes SQL with ? placeholders; transaction helper
Database	MySQL 8.0 (InnoDB)	Storage, integrity constraints, transactions, indexes, view
Security	JSON Web Tokens, bcrypt	Stateless login, password hashing

Why raw SQL instead of an ORM? This is a DBMS project, so every query is written in plain SQL and stays visible in the code, which keeps each JOIN, GROUP BY and transaction easy to explain. All user values are passed as ? placeholders, which prevents SQL injection.

12.2 System architecture



All data flows through SQL. The browser never talks to MySQL directly; the API validates input, checks the JWT and role, then runs parameterised SQL.

During development Vite forwards every /api request to the Express server (proxy in vite.config.js). For production, npm start builds the React app and Express serves it from the same port.

```

export default defineConfig({
  plugins: [react()],
  server: {
    port: 5173,
    // Forward /api calls to the Express backend during development
    proxy: {
      '/api': { target: 'http://localhost:5000', changeOrigin: true },
    },
  },
})

```

12.3 Project structure and size

```

retaildesk/
|-- package.json          root scripts: install:all, setup-db, dev, start, test:api
|-- README.md
- database/
  |-- schema.sql          DDL: tables, keys, constraints, indexes, view
  |-- seed.sql             sample data (dates relative to today)
  |-- queries.sql          22 demo queries (also used by the Database page)
  |-- crud_examples.sql    INSERT / UPDATE / DELETE + transaction examples
-- server/
  |-- server.js            Express app
  |-- config/              env + MySQL connection pool + transaction helper
  |-- routes/index.js      all REST routes
  |-- controllers/         one file per module (SQL lives here)
  |-- services/            saleService (transaction), queryLibrary (parses queries.sql)
  |-- middleware/          JWT auth, roles, MySQL-error -> friendly-message handler
  |-- utils/               validation, pagination, errors
  |-- scripts/             setup-db.js, generate-seed.js, api-test.js
-- client/
  |-- vite.config.js       /api proxy -> :5000
  -- src/
    |-- App.jsx            routes
    |-- layouts/           sidebar + top bar
    |-- pages/             Dashboard, Products, ..., DatabasePage, Settings
    |-- components/        UI kit, ER diagram, SQL code viewer, charts
    |-- context/           auth, toasts, store settings
    |-- hooks/             data fetching, list state
    |-- services/api.js    fetch wrapper
    |-- utils/             formatting (INR, dates), CSV export

```

Table 17: Source code size (excluding node_modules)

Component	Files	Lines
Database (SQL scripts)	4	1,217
Backend (Node.js / Express)	26	2,721
Frontend pages and components (React)	33	4,408
Styling and configuration	9	3,337
Total	72	11,683

12.4 Database connectivity

Connection settings come from `server/.env` (copied from `.env.example`). The server creates one **connection pool** of up to 10 connections, so concurrent requests do not open a new connection each time. DECIMAL values are returned as JavaScript numbers. The helper with `Transaction` obtains a connection, begins a transaction, commits when the work function succeeds and rolls back when it throws.

Table 18: Environment variables (server/.env)

Variable	Example	Description
DB_HOST / DB_PORT	localhost / 3306	MySQL host and port
DB_USER / DB_PASSWORD	root / (secret)	MySQL credentials
DB_NAME	retail_inventory_db	Database name created by setup-db
DATABASE_URL	mysql://user:pass@host:3306/db	Optional; overrides the DB_* values
PORT, CLIENT_ORIGIN	5000, http://localhost:5173	API port and allowed CORS origin
JWT_SECRET, JWT_EXPIRES_IN	(long random string), 12h	Token signing secret and session length

server/config/db.js

```

import mysql from 'mysql2/promise';
import { getDbConfig } from './env.js';

export const dbConfig = getDbConfig();

export const pool = mysql.createPool({
  ...dbConfig,
  waitForConnections: true,
  connectionLimit: 10,
});

```

```

    decimalNumbers: true, // DECIMAL columns and SUM() results come back as JS numbers
    supportBigNumbers: true,
  });

  /** Run a single statement on the pool and return the rows (or the result header). */
  export async function query(sql, params = []) {
    const [rows] = await pool.query(sql, params);
    return rows;
  }

  /** Return the first row or null. */
  export async function queryOne(sql, params = []) {
    const rows = await query(sql, params);
    return rows[0] ?? null;
  }

  /**
   * Run `work(conn)` inside a database transaction.
   * COMMIT if it resolves, ROLLBACK if anything throws.
   */
  export async function withTransaction(work) {
    const conn = await pool.getConnection();
    try {
      await conn.beginTransaction();
      const result = await work(conn);
      await conn.commit();
      return result;
    } catch (err) {
      await conn.rollback();
      throw err;
    } finally {
      conn.release();
    }
  }
}

```

12.5 Authentication and authorisation

POST/api/auth/login checks the password with bcrypt and returns a signed JWT. All other routes (except /health) sit behind requireAuth; destructive or sensitive routes additionally use requireRole. The React app stores the token and sends it as Authorization: Bearer <token>; a 401 response logs the user out automatically.

server/middleware/auth.js

```

import jwt from 'jsonwebtoken';
import { env } from '../config/env.js';
import { ApiError } from '../utils/ApiError.js';

/** Require a valid "Authorization: Bearer <token>" header. */
export function requireAuth(req, res, next) {
  const header = req.headers.authorization || '';
  const token = header.startsWith('Bearer ') ? header.slice(7) : null;
  if (!token) return next(ApiError.unauthorized());
  try {
    req.user = jwt.verify(token, env.jwtSecret);
    next();
  } catch {
    next(ApiError.unauthorized('Your session has expired. Please sign in again.'));
  }
}

/** Restrict a route to the given roles. */
export const requireRole = (...roles) => (req, res, next) =>
  roles.includes(req.user?.role)
    ? next()
    : next(new ApiError(403, 'You do not have permission to perform this action'));

```

server/routes/index.js (excerpt)

```

router.get('/products', h(products.listProducts));
router.get('/products/:id', h(products.getProduct));
router.post('/products', managers, h(products.createProduct));
router.put('/products/:id', managers, h(products.updateProduct));
router.delete('/products/:id', managers, h(products.deleteProduct));
...
router.get('/sales', h(sales.listSales));
router.get('/sales/:id', h(sales.getSale));
router.post('/sales', h(sales.postSale));

```

12.6 The transactional sale (key code)

Creating a sale is the most important operation of the system. `createSale` runs inside `withTransaction`: it locks the stock rows with `SELECT ... FOR UPDATE`, validates stock, computes money on the server (never trusting client totals; amounts are handled in paise to avoid floating-point errors), inserts the sale, items, stock reduction, audit rows and the payment, and commits. Any thrown error triggers `ROLLBACK`.

server/services/saleService.js (excerpt)

```
const saleId=await withTransaction(async (conn) => {
  const settings = await getSettings(conn);

  if (customerId) {
    const [[customer]] = await conn.query('SELECT id FROM customers WHERE id = ?', [customerId]);
    if(!customer) throw ApiError.badRequest('Selected customer does not exist');
  }

  // 1. Lock stock rows (row-level locks prevent two cashiers overselling the same item)
  const ids= items.map((i) => i.product_id);
  const [rows]= await conn.query(
    `SELECT p.id,p.product_name, p.selling_price, i.quantity AS stock
    FROM products p
    JOIN inventory i ON i.product_id = p.id
    WHERE p.id IN (?)
    FOR UPDATE`,
    [ids],
  );
  ...
  // 3. Sale header
  const [saleResult] = await conn.query(
    `INSERT INTO sales (customer_id, user_id, subtotal, discount, tax, total_amount, payment_status, payment_method,
    notes)
    VALUES(?, ?, ?, ?, ?, ?, ?, ?)`,
    [customerId, userId, toRupees(subtotal), toRupees(discount), toRupees(tax), toRupees(total), status, method, notes],
  );
  const id=saleResult.insertId;
  const invoiceNo= invoiceNumber(settings.invoice_prefix, id);
  await conn.query('UPDATE sales SET invoice_no = ? WHERE id = ?', [invoiceNo, id]);

  // 4. Line items (bulk insert; subtotal is a generated column)
  await conn.query('INSERT INTO sale_items (sale_id, product_id, quantity, unit_price) VALUES ?', [
    lines.map((l) => [id, l.product_id, l.quantity, toRupees(l.price)]),
  ]);

  // 5. Reduce stock – the "quantity >= ?" guard is a second line of defence against overselling
  for (const line of lines) {
    const [upd]=await conn.query(
      `UPDATE inventory SET quantity = quantity - ? WHERE product_id = ? AND quantity >= ?`,
      [line.quantity, line.product_id, line.quantity],
    );
    if(upd.affectedRows !== 1) throw ApiError.conflict(`Stock for ${line.name} changed. Please try again.`);
    await conn.query(
      `INSERT INTO stock_movements (product_id, user_id, sale_id, movement_type, quantity_change, quantity_after,
      reason)
      VALUES (?, ?, ?, 'SALE', ?, ?, ?)`,
      [line.product_id, userId, id, -line.quantity, line.stock - line.quantity, `Sold on ${invoiceNo}`],
    );
  }

  // 6. Payment record
  const ref = method === 'Cash' ? null : `${method.slice(0, 3).toUpperCase()}${Date.now().toString().slice(-8)}`;
  await conn.query(
    `INSERT INTO payments (sale_id, amount, payment_method, payment_status, transaction_ref) VALUES (?, ?, ?, ?, ?)`,
    [id, toRupees(total), method, status, ref],
  );

  return id;
});
```

12.7 Front-end data layer

Pages never call `fetch` directly. A small wrapper adds the token, converts errors to friendly messages and logs the user out on 401; the `useApi` hook re-fetches when its parameters change and ignores out-of-date responses.

client/src/services/api.js (excerpt)

```
/** Fetch wrapper: JSON in/out, bearer token, friendly errors, auto-logout on 401. */
export async function request(path, { method = 'GET', body, params } = {}) {
  const headers = { Accept: 'application/json' };
```

```

const token = tokenStore.get();
if (token) headers.Authorization = `Bearer ${token}`;
if (body !== undefined) headers['Content-Type'] = 'application/json';

let res;
try {
  res = await fetch(`${BASE_URL}${path}${buildQuery(params)}`, {
    method,
    headers,
    body: body !== undefined ? JSON.stringify(body) : undefined,
  });
} catch {
  throw new ApiError('Cannot reach the server. Is the backend running on port 5000?', 0);
}

let data = null;
try { data = await res.json(); } catch { /* empty body */ }

if (!res.ok) {
  if (res.status === 401 && path !== '/auth/login') {
    tokenStore.clear();
    window.dispatchEvent(new Event('auth:logout'));
  }
  throw new ApiError(data?.message || `Request failed (${res.status})`, res.status, data?.errors);
}
return data;
}

export const api = {
  get: (path, params) => request(path, { params }),
  post: (path, body) => request(path, { method: 'POST', body: body ?? {} }),
  put: (path, body) => request(path, { method: 'PUT', body: body ?? {} }),
  delete: (path) => request(path, { method: 'DELETE' }),
};

```

12.8 REST API summary

Base URL <http://localhost:5000/api>. List endpoints accept `?page=&limit=&search=&sort=&order=` and return `{data,pagination }`. Errors return `{success: false, message, errors? }` with status 400 / 401 / 403 / 404 / 409.

Table 19: API endpoints

Method	Endpoint	Description
GET	/health	Database connectivity check
POST	/auth/login	Email + password → token and user
GET	/dashboard	All dashboard statistics, trends and lists
GET / POST	/products	List with filters / create product and its inventory row (transaction)
GET / PUT / DELETE	/products/:id	Details with stats / update / delete (409 if the product has sales)
GET / POST	/categories, /suppliers, /customers	List (with counts or totals) / create
GET / PUT / DELETE	/categories/:id, /suppliers/:id, /customers/:id	Details / update / delete (409 if in use)
GET	/inventory, /inventory/movements	Stock list with summary; stock audit log
PUT	/inventory/:productId	Adjust stock { type: add remove set, quantity, reason }
GET / POST	/sales	List with filters / transactional sale
GET	/sales/:id	Sale with customer, items, payments and store info
GET / PUT	/payments, /payments/:id	List with summary / update a pending payment
GET	/reports, /reports/:type	Nine reports with date filter
GET	/database/schema, /database/queries	Live schema from INFORMATION_SCHEMA; predefined queries
POST	/database/queries/:id/run, /database/transaction-demo	Run a predefined query read-only; live ROLLBACK demonstration
GET / PUT	/settings	Store settings (PUT: Admin only)
GET	/search?q=, /notifications	Global search; low-stock and pending-payment alerts

12.9 Security and data-integrity measures

- Passwords stored only as **bcrypt hashes**; JWT secret and database password kept in server/.env (excluded from version control).
- **Parameterised queries** everywhere (? placeholders) against SQL injection; the SQL demo never executes SQL sent by the browser.
- **Role-based access** (Admin, Manager, Cashier) checked on the server for every sensitive route.
- Server-side validation (required fields, formats, non-negative numbers) in addition to database CHECK / UNIQUE / FOREIGN KEY constraints.
- MySQL errors are mapped to friendly messages (duplicate key → 409, constraint → 400, foreign-key conflict → 409).
- **Row-level locking** (FOR UPDATE) plus a guarded UPDATE ... WHERE quantity >= ? prevent overselling under concurrency.

13. Testing (Test Cases and Results)

13.1 Test strategy

Testing combined three levels: (1) **constraint tests** directly in SQL (Section 9.4), (2) **query tests**, where all 22 demonstration queries were executed (Section 10), and (3) an **automated end-to-end API test suite** (server/scripts/api-test.js) that exercises the real HTTP API against a real MySQL database. The suite creates its own uniquely named records, so it can run repeatedly.

Test environment: Windows 11, Node.js, MySQL 8.0.46, a freshly created copy of the database loaded from schema.sql and seed.sql (a separate scratch database was used so that the application's own data was not changed). **Run date:** 5 October 2026.

13.2 Summary of results

Table 20: Automated test results by module

Module	Test cases	Passed	Failed
Health & authentication	5	5	0
Read endpoints	14	14	0
Category CRUD + referential integrity	5	5	0
Supplier CRUD	4	4	0
Customer CRUD	3	3	0
Product CRUD + validation	7	7	0
Inventory adjustment	3	3	0
Sale transaction	11	11	0
Stock validation	5	5	0
Payments	6	6	0
Reports	10	10	0
Database / SQL demonstration	27	27	0
Settings	2	2	0
Role permissions	2	2	0
Delete + cleanup	7	7	0
Total	111	111	0

Result: 111 passed, 0 failed (command output: 111 passed, 0 failed). The most important checks demonstrate the DBMS concepts: constraint errors are returned as 4xx responses, a sale decreases stock exactly once, a failed sale leaves stock unchanged (rollback), overselling is refused, and referential actions behave as designed.

13.3 Test cases and results

Each case below states the action and the expected outcome (for example an expected HTTP status). All cases passed.

Table 21: Complete list of automated test cases

ID	Module	Test case and expected result	Result
TC001	Health & authentication	GET /health → database reachable	Pass
TC002	Health & authentication	Protected route without token → 401	Pass
TC003	Health & authentication	Login with wrong password → 401	Pass
TC004	Health & authentication	Login admin@retail.com / admin123 → token	Pass
TC005	Health & authentication	GET /auth/me	Pass
TC006	Read endpoints	GET /products → 200	Pass
TC007	Read endpoints	GET /categories → 200	Pass
TC008	Read endpoints	GET /suppliers → 200	Pass
TC009	Read endpoints	GET /customers → 200	Pass
TC010	Read endpoints	GET /inventory → 200	Pass
TC011	Read endpoints	GET /inventory/movements → 200	Pass
TC012	Read endpoints	GET /sales → 200	Pass
TC013	Read endpoints	GET /payments → 200	Pass
TC014	Read endpoints	GET /settings → 200	Pass
TC015	Read endpoints	GET /notifications → 200	Pass
TC016	Read endpoints	GET /search?q=rice → 200	Pass
TC017	Read endpoints	GET /reports → 200	Pass
TC018	Read endpoints	Products list is paginated	Pass
TC019	Read endpoints	Filter products by status	Pass
TC020	Category CRUD + referential integrity	POST /categories → 201	Pass
TC021	Category CRUD + referential integrity	Duplicate category name → 409	Pass
TC022	Category CRUD + referential integrity	Empty category name → 400	Pass
TC023	Category CRUD + referential integrity	PUT /categories/:id	Pass
TC024	Category CRUD + referential integrity	Delete category that has products → 409 (ON DELETE RESTRICT)	Pass
TC025	Supplier CRUD	POST /suppliers → 201	Pass
TC026	Supplier CRUD	Invalid supplier email → 400	Pass
TC027	Supplier CRUD	Invalid supplier phone → 400	Pass
TC028	Supplier CRUD	GET /suppliers/:id includes products	Pass
TC029	Customer CRUD	POST /customers → 201	Pass
TC030	Customer CRUD	Duplicate customer phone → 409	Pass
TC031	Customer CRUD	PUT /customers/:id	Pass
TC032	Product CRUD + validation	POST /products (with opening stock 50) → 201	Pass
TC033	Product CRUD + validation	New product has inventory row with 50 units	Pass
TC034	Product CRUD + validation	Duplicate SKU → 409	Pass
TC035	Product CRUD + validation	Negative price → 400	Pass
TC036	Product CRUD + validation	Missing product name → 400	Pass
TC037	Product CRUD + validation	Non-existent category → 400	Pass
TC038	Product CRUD + validation	PUT /products/:id	Pass

ID	Module	Test case and expected result	Result
TC039	Inventory adjustment	Stock in +10 → 60	Pass
TC040	Inventory adjustment	Remove more than in stock → 400	Pass
TC041	Inventory adjustment	Negative quantity → 400	Pass
TC042	Sale transaction	POST /sales → 201	Pass
TC043	Sale transaction	Invoice number generated	Pass
TC044	Sale transaction	Subtotal = 3 × 160 = 480	Pass
TC045	Sale transaction	Tax computed on server (84.6)	Pass
TC046	Sale transaction	Total = subtotal – discount + tax	Pass
TC047	Sale transaction	1 sale item and 1 payment written	Pass
TC048	Sale transaction	Inventory decreased 60 → 57	Pass
TC049	Sale transaction	Stock movement logged for the sale	Pass
TC050	Sale transaction	Dashboard total orders +1	Pass
TC051	Sale transaction	Dashboard today's sales increased by the sale total	Pass
TC052	Sale transaction	Customer history shows the sale	Pass
TC053	Stock validation	Selling more than available → 400	Pass
TC054	Stock validation	Failed sale did not change stock (rollback)	Pass
TC055	Stock validation	Selling an out-of-stock product → 400	Pass
TC056	Stock validation	Empty cart → 400	Pass
TC057	Stock validation	Discount greater than subtotal → 400	Pass
TC058	Payments	Pending sale created	Pass
TC059	Payments	Mark pending payment as Paid → sale becomes Paid	Pass
TC060	Payments	Changing an already-paid payment → 400	Pass
TC061	Payments	Filter payments by status	Pass
TC062	Payments	GET /sales/:id	Pass
TC063	Payments	Filter sales by payment method	Pass
TC064	Reports	GET /reports/daily-sales (14 rows)	Pass
TC065	Reports	GET /reports/weekly-sales (6 rows)	Pass
TC066	Reports	GET /reports/monthly-sales (2 rows)	Pass
TC067	Reports	GET /reports/product-sales (33 rows)	Pass
TC068	Reports	GET /reports/category-sales (14 rows)	Pass
TC069	Reports	GET /reports/customer-purchases (18 rows)	Pass
TC070	Reports	GET /reports/inventory (33 rows)	Pass
TC071	Reports	GET /reports/low-stock (8 rows)	Pass
TC072	Reports	GET /reports/suppliers (9 rows)	Pass
TC073	Reports	Invalid report date → 400	Pass
TC074	Database / SQL demonstration	Schema lists 11 tables	Pass
	Database / SQL demonstration		
TC075	Database / SQL demonstration	Schema lists 11 foreign keys	Pass
TC076	Database / SQL demonstration	22 predefined queries loaded	Pass
TC077	demonstration	Run "Products with category names" (33 rows)	Pass

ID	Module	Test case and expected result	Result
TC078	Database / SQL demonstration	Run "Products with supplier names" (33 rows)	Pass
TC079	Database / SQL demonstration	Run "Products at or below reorder level" (8 rows)	Pass
TC080	Database / SQL demonstration	Run "Overall sales summary" (1 rows)	Pass
TC081	Database / SQL demonstration	Run "Today's sales" (6 rows)	Pass
TC082	Database / SQL demonstration	Run "Top 10 selling products (by quantity)" (10 rows)	Pass
TC083	Database / SQL demonstration	Run "Monthly sales" (4 rows)	Pass
TC084	Database / SQL demonstration	Run "Total sales by category" (13 rows)	Pass
TC085	Database / SQL demonstration	Run "Customers with the highest purchase amounts" (10 rows)	Pass
TC086	Database / SQL demonstration	Run "Total inventory value (with ROLLUP)" (14 rows)	Pass
TC087	Database / SQL demonstration	Run "Products that have never been sold" (2 rows)	Pass
TC088	Database / SQL demonstration	Run "Best-selling products (HAVING filter)" (15 rows)	Pass
TC089	Database / SQL demonstration	Run "Sales with customer and cashier details" (25 rows)	Pass
TC090	Database / SQL demonstration	Run "Total quantity sold per product (including zero)" (33 rows)	Pass
TC091	Database / SQL demonstration	Run "Suppliers supplying the most products" (9 rows)	Pass
TC092	Database / SQL demonstration	Run "Customers who spend more than the average customer" (9 rows)	Pass
TC093	Database / SQL demonstration	Run "Payment method & status summary" (6 rows)	Pass
TC094	Database / SQL demonstration	Run "Daily sales for the last 7 days" (5 rows)	Pass
TC095	Database / SQL demonstration	Run "Querying a VIEW (v_product_stock)" (8 rows)	Pass
TC096	Database / SQL demonstration	Run "Categories with product counts (including empty)" (14 rows)	Pass
TC097	Database / SQL demonstration	Run "Rank products by revenue within each category" (33 rows)	Pass
TC098	Database / SQL demonstration	Run "Stock movement audit trail" (25 rows)	Pass
TC099	Database / SQL demonstration	Unknown query id → 404	Pass
TC100	Settings Settings	Transaction rollback demo keeps data consistent	Pass
TC101	Role permissions	Invalid tax rate → 400	Pass
TC102	Role permissions	PUT /settings	Pass
TC103		Cashier cannot delete products → 403	Pass
TC104		Cashier can view dashboard	Pass

ID	Module	Test case and expected result	Result
TC105	Delete + cleanup	Product with sales cannot be deleted → 409	Pass
TC106	Delete + cleanup	Delete customer (sales kept as walk-in)	Pass
TC107	Delete + cleanup	Sale now shows walk-in customer	Pass
TC108	Delete + cleanup	Delete supplier (product supplier set NULL)	Pass
TC109	Delete + cleanup	Product kept with supplier_id NULL	Pass
TC110	Delete + cleanup	Delete unsold product → 200 (inventory row cascades)	Pass
TC111	Delete + cleanup	Deleted product → 404	Pass

13.4 Additional verification

Table 22: Manual and demonstration checks

Check	Procedure	Observed result
Live ROLLBACK demo	Database → Transactions → Run demo	Sale inserted, stock reduced, oversell attempt rejected by CHECK (error 3819), ROLLBACK issued; sales count 51 → 51 and stock 120 → 120 (Figure 16).
Stock update after sale	SQL transaction on product #1 (stock 48), sell 1 unit, then ROLLBACK	48 → 47 inside the transaction → 48 after rollback (Section 9.5).
Restrict on delete	DELETE category "Groceries" (in use)	Error 1451, foreign key constraint fails (Section 9.4).
Duplicate SKU	INSERT product with existing SKU	Error 1062, duplicate entry (Section 9.4).
Query execution	All 22 queries run through the API test suite; query 06 also run from the	All 22 returned results with no error; outputs are listed in Section 10.
Dashboard consistency	Database page (Figure 15) Compare dashboard KPIs with direct SQL	Orders 51, customers 26, products 32, revenue ₹80,332 (SUM of total_amount = 80332.04) and inventory value ₹2,61,339 (SUM of quantity × purchase_price = 261339.00) are identical to the database.
Invoice arithmetic	Open invoice INV-000051 (Figure 9)	Subtotal ₹1,898.00 + GST 18% ₹341.64 = grand total ₹2,239.64, as expected.

14. Conclusion and Future Enhancements

14.1 Conclusion

The project achieved its objective: a complete, working retail inventory and sales management system built around a well-designed relational database. The schema of 11 tables and one view is in Third Normal Form, protected by primary, foreign and unique keys, CHECK constraints, a generated column and purposeful referential actions. The most important business operation, recording a sale, is a single ACID transaction that locks stock rows, validates availability, writes five tables and rolls back completely on any error. Twenty-two SQL queries demonstrate joins, aggregates, GROUP BY / HAVING, subqueries, ROLLUP, a view and a window function, and the web interface turns these queries into dashboards, reports, invoices and a POS screen. The automated suite of 111 API checks passed in full, which gives confidence that the database rules and the application behave as designed.

Working on the project reinforced that integrity belongs in the database: the CHECK, UNIQUE and FOREIGN KEY constraints caught invalid data even when the application layer was bypassed, and keeping the SQL visible (no ORM) made each design decision easy to reason about and to explain.

14.2 Limitations

- Single store and single currency (INR); one GST rate for the whole store.
- Payments are recorded but not integrated with a real payment gateway.
- No returns or refunds; no purchase-order workflow, so restocking is a manual stock adjustment.
- Automated tests cover the API and database; the React front end has no unit tests yet.

14.3 Future enhancements

- Purchase orders and goods-received notes (automatic restock from suppliers).
- Barcode scanning in the POS and barcode label printing.
- Returns / refunds with reverse stock movements.
- Batch and expiry-date tracking for groceries (FEFO).
- Multi-store / multi-warehouse inventory.
- Per-product GST slabs (0%, 5%, 12%, 18%, 28%) and GST reports (GSTR-1).
- Customer loyalty points and SMS / WhatsApp e-invoices.
- Stored procedures and triggers for low-stock alerts.
- Scheduled database backups and an audit log for all edits.
- Front-end unit tests and a continuous-integration pipeline.

15. References

- [1] OracleCorporation. *MySQL 8.0 Reference Manual* (InnoDB, constraints, transactions, window functions, generated columns). <https://dev.mysql.com/doc/refman/8.0/en/>
- [2] Elmasri, R. and Navathe, S. B. *Fundamentals of Database Systems*, 7th ed. Pearson, 2016.
- [3] Silberschatz, A., Korth, H. F. and Sudarshan, S. *Database System Concepts*, 7th ed. McGraw-Hill, 2019.
- [4] Codd, E. F. "A Relational Model of Data for Large Shared Data Banks." *Communications of the ACM* 13(6), 1970.
- [5] Chen, P. P. "The Entity-Relationship Model: Toward a Unified View of Data." *ACM Transactions on Database Systems* 1(1), 1976.
- [6] Express.js documentation. <https://expressjs.com/>
- [7] Node.js documentation. <https://nodejs.org/docs/latest/api/>
- [8] node-mysql2 (mysql2) driver. <https://github.com/sidorares/node-mysql2>
- [9] React documentation. <https://react.dev/>
- [10] Vite documentation. <https://vitejs.dev/>
- [11] Tailwind CSS documentation. <https://tailwindcss.com/docs>
- [12] Recharts documentation. <https://recharts.org/>
- [13] Jones, M., Bradley, J. and Sakimura, N. *JSON Web Token (JWT)*, RFC 7519, IETF, 2015.
<https://datatracker.ietf.org/doc/html/rfc7519>
- [14] bcrypt.js: password hashing for JavaScript. <https://github.com/dcodeIO/bcrypt.js>
- [15] Project README and SQL scripts of the RetailDesk repository (README.md, database/*.sql).

16. Contribution of Each Member

Fill in the table below with the actual contribution of each group member. The areas of work in this project, which can be used as a guide, are: database design and ER diagram; SQL scripts and queries; back-end API and transactions; front-end screens; testing; and documentation.

Table 23: Contribution of group members

Member	Roll no.	Contribution
[Member 1 name]	[Roll no.]	[e.g. ER diagram, normalisation, schema.sql, data dictionary]
[Member 2 name]	[Roll no.]	[e.g. SQL queries, transaction logic, back-end API]
[Member 3 name]	[Roll no.]	[e.g. front-end screens, testing, report]

17. Appendix: GitHub Repository Link

Table 24: Repository

Item	Value
GitHub repository	[<a href="https://github.com/<username>/<repository>">https://github.com/<username>/<repository>]
Branch / commit submitted	[main / commit id]
Setup (from the README)	npm run install:all → copy server\.env.example to server\.env and set DB_PASSWORD → npm run setup-db → npm run dev → open http://localhost:5173
Demo login	admin@retail.com / admin123 (Manager: manager@retail.com / manager123, Cashier: cashier@retail.com / cashier123)
Automated tests	npm run test:api (with the server running): 111 checks

Appendix A: Demonstration SQL files in the repository

File	Lines	Content
database/schema.sql	267	DDL: database, 11 tables, constraints, indexes, view v_product_stock
database/seed.sql	584	Sample data (settings, users, categories, suppliers, products, inventory, customers, sales, items, payments, movements)
database/queries.sql	274	22 demonstration queries with metadata (id, title, concepts, description)
database/crud_examples.sql	92	INSERT / UPDATE / DELETE, constraint violations and the sale transaction